

Politecnico di Torino

Master's degree in Cybersecurity

A.Y. 2025/2026

Graduation Session September 2026

Vuln-Forge: Creating Containerized Vulnerable Environments with LLM-based Agents

Supervisor: Prof. Matteo Boffa

Candidate: Manuele Sforzini

Abstract

Large Language Models (LLMs) are nowadays showing increasing capabilities in complex technical tasks. Cybersecurity is one of the domains in which LLMs' potential generates significant expectations and, most importantly, given its dual-use nature, concerns.

One of the most extensively tested capabilities is their ability to penetrate vulnerable systems, typically evaluated through Capture The Flag (CTF) challenges in which an agent must exploit a vulnerability in a controlled environment to retrieve a hidden flag. However, building such environments requires substantial engineering and human effort, making these benchmarks difficult to scale with the growing number of newly discovered vulnerabilities. This limits our ability to assess LLM agents against recent attacks and, as models continue to evolve, introduces the risk of benchmark aging, as existing benchmarks may enter models' training data and eventually become saturated.

In this thesis, I investigate whether the generation of vulnerable playground environments can be automated using agentic AI. To this end, I designed and implemented a *forger agent* that, given a CVE description, generates a containerized environment exposing the corresponding vulnerability and places a flag in a location that can only be reached by successfully exploiting it. This requires the agent to i) identify the vulnerable service and configuration, ii) reproduce them in a working container, and iii) make the resulting environment exploitable. I evaluated the forger on 226 CVEs, selected across 25 vulnerability types, different release periods, and difficulty levels. As a first validation method, I used Docker Scout, which statically checks whether the generated container includes software versions associated with the target CVE. Since static analysis alone cannot determine whether a vulnerability is actually exploitable, I complemented this evaluation with dynamic validation. For 94 CVEs with a public Proof of Concept (PoC), I executed the known exploit against the generated environment and considered the generation successful when the exploit retrieved the flag. To extend dynamic validation to the remaining CVEs, for which no public PoC is available, I also designed an *evaluator agent*. Given the CVE description and the environment generated by the forger, the evaluator actively analyzes and attacks the container, synthesizing an exploit that attempts to retrieve the flag. Rather than restricting it to the CVEs without a PoC, I run the evaluator against every generated environment, providing an independent cross-check on the outcome of the PoC-based validation.

The results show that static validation alone is not sufficient to assess the generated environments: Docker Scout misses many environments that are successfully

exploited by the corresponding PoC. The evaluator recovers most of these cases by successfully retrieving the flag where Docker Scout reports no vulnerability. Conversely, the cases identified only by Docker Scout are not confirmed by the PoC, showing that dynamic exploitation provides a more reliable validation signal. Overall, these results demonstrate the feasibility of using agentic AI not only to automatically generate vulnerable playgrounds, but also to dynamically evaluate whether the generated vulnerabilities are actually exploitable.

A Ilaria, che ha reso speciale ogni singolo istante di questo percorso. Non ce l'avrei mai fatta senza di te, questo traguardo è anche un po' tuo.

Table of Contents

List of Tables IX

List of Figures XI

List of Acronyms XIII

1 Introduction 1

2 Background and Related Work 5
2.1 Software Security: Definitions 5
2.1.1 Vulnerabilities 5
2.1.2 Common Vulnerabilities and Exposures (CVE) 6
2.1.3 Common Weakness Enumeration (CWE) 7
2.1.4 Related Metrics: CVSS and EPSS 7
2.1.5 Exploit and Proof of Concept 8
2.1.6 Software Patches 8
2.1.7 Attack Vectors 9
2.1.8 Kill Chain 9
2.1.9 Selected Vulnerability Classes 10
2.2 Virtualization and Containerization 12
2.2.1 Virtualization 13
2.2.2 Lightweight Virtualization and Containers 13
2.2.3 Docker 13
2.2.4 Container Networking 14
2.3 LLMs and AI Agents 15
2.3.1 Artificial Intelligence and Machine Learning 15
2.3.2 Large Language Models and Prompts 16
2.3.3 LLMs and AI Agents 18
2.3.4 Multi-Agent Systems 18
2.4 Security Testing and Validation Methodology 18
2.4.1 Static and Dynamic Analysis 19

2.4.2 Testing Oracles . . . 19
2.4.3 Ground Truth and Evaluation Metrics . . . 19
2.4.4 Capture The Flag (CTF) . . . 20
2.4.5 Penetration Testing, Red Teaming and Blue Teaming . . . 20
2.5 Related Work . . . 20
2.5.1 Curated Vulnerable Environments . . . 20
2.5.2 Automated Vulnerable-Environment Generation . . . 21
2.5.3 Benchmarks for LLM-Based Cybersecurity Agents . . . 22
2.5.4 Autonomous Penetration Testing . . . 22
2.5.5 Positioning of the Proposed Work . . . 23

3 Methodology 24
3.1 Pipeline Overview . . . 26
3.2 Building the Environment: the Forger Agent . . . 27
3.2.1 Web Search and Platform Feasibility Check . . . 28
3.2.2 Vulnerability Classification and Flag Placement . . . 29
3.2.3 Environment Code Generation . . . 31
3.2.4 Iterative Testing and Revision: Creating a Working Environment . . . 34
3.3 Sanity Check: Verifying if the Flag is Exposed . . . 37
3.4 Vulnerability Validation: Is the Environment Exploitable? . . . 38
3.4.1 Static Validation Baseline . . . 38
3.4.2 Dynamic Validation: Red Teaming . . . 39
3.5 Debugging After a Failed Exploit . . . 42
3.6 PoC-Based Validation . . . 43
3.6.1 Retrieving the PoCs: The Scraper . . . 43
3.6.2 PoC Adaptation and Execution . . . 44

4 System Design and Implementation 45
4.1 System Architecture . . . 45
4.2 Agent and Workflow Implementation . . . 46
4.2.1 The Graph . . . 46
4.2.2 Partial Executions and Milestones Reset . . . 46
4.2.3 Running the Evaluator . . . 47
4.3 State Representation and Structured Outputs . . . 48
4.4 Docker Orchestration . . . 48
4.5 Evaluator Isolation . . . 49
4.6 Nonce Injection . . . 50
4.7 Automatic Reconnaissance . . . 50
4.8 Prompt Design . . . 51
4.9 Logging and Results Persistence . . . 51

4.10 PoC-Scraper Implementation . . . 52
4.11 PoC-Runner Implementation . . . 53

5 Results 55
5.1 Dataset Overview . . . 55
5.2 Comparison between Docker Scout, the Evaluator and the PoC . . . 58
5.3 Comparison between Docker Scout and the Evaluator . . . 60
5.4 Forger Effectiveness based on the CWE . . . 60
5.5 Evaluator Effectiveness based on the CWE . . . 64
5.6 Revision Loop and Debugger Loop Effectiveness . . . 66
5.6.1 Impact of the Revision Loop . . . 66
5.6.2 Impact of the Debugging Loop . . . 69
5.7 Other Metrics . . . 71
5.7.1 Success Rate per CVE Release Period . . . 71
5.7.2 Success Rate per Container Number . . . 73
5.7.3 Success Rate per Kill Chain Length . . . 75
5.7.4 Success Rate per Attack Vector (Network vs Local) . . . 76
5.7.5 Success Rate per Techstack . . . 76
5.7.6 Success Rate per EPSS Score (CVE Virality) . . . 77

6 Conclusions and Future Work 80
6.1 Summary of Contributions . . . 80
6.2 Limitations . . . 81
6.3 Future Work . . . 83

A Some Relevant Code Definitions 86
A.1 VulnerabilityClass . . . 86
A.2 OverallState . . . 86
A.3 CVEClassification . . . 89
A.4 Milestones . . . 90
A.5 Stats . . . 91
A.6 DebuggerDecision . . . 91

Bibliography 93

List of Tables

2.1 Comparison between representative related approaches and the proposed work. 23

3.1 Summary of the validation strategies adopted in this thesis 25

5.1 Mapping between CWE-ID and CWE-Name, along with the number of CVEs for each CWE in the *Forger Dataset* (FD) and in the *Evaluator Dataset* (ED) 57
5.2 Comparison between static validation (Docker Scout) and the public PoC adapted. 58
5.3 Comparison between static validation (Docker Scout) and dynamic exploitation outcome (*evaluator*) on the 94 generated environments for which a PoC was available 58
5.4 Comparison between the *evaluator* and the public PoC adapted. . . 59
5.5 Comparison between static validation (Docker Scout) and dynamic exploitation outcome (Evaluator) on all the 150 generated environments 60
5.6 Environment generation results for the most represented CWE classes 62
5.7 Environment generation results for the least represented CWE classes (n<8), grouped by the vulnerability family. The reported values do not consider the results already discussed in Table 5.6. 63
5.8 Exploit success rate for the most represented classes, among the environments tested by the *evaluator* (n=150). 64
5.9 Exploit success rate for the least represented classes (n<8 in the main dataset), grouped by their vulnerability family, among the environments in which the *evaluator* was executed. These results are computed excluding the values already present in Table 5.8. . . 65
5.10 Failure reasons in the revision loop, considering the full CVE dataset. 66
5.11 Average iteration number for each environment (including iterations executed before a *Hard Fix*) with respect to the kill chain length. . 67
5.12 Error pattern in the environments in the *Forger Dataset* that are not in the *Evaluator Dataset*. 68

5.13 Frequency of debug-loop usage and final success rate, at the environment level (N = 150) 69
5.14 Immediate outcome of each debugger invocation, by fix type. 70
5.15 Environment generation success rate per CVE release period. 72
5.16 Success rate per CVE release period. 72
5.17 Environment generation success rate per container number in the environment. 73
5.18 Success rate per container number in the environment. 74
5.19 Environment generation success rate per kill chain length. 75
5.20 Success rate per kill chain length. 75
5.21 Success rate per attack vector. In this table are reported both the performances of the *forger* and of the *evaluator* 76
5.22 Environment generation success rate per underlying techstack. . . . 77
5.23 Exploit success rate per underlying techstack. 77
5.24 Distribution of the generated environments according to their EPSS score. In this table are reported both the performances of the *forger* and of the *evaluator* 78

List of Figures

3.1 The methodology adopted to create and validate vulnerable environments. 24
3.2 The pipeline used to generate vulnerable environments 26
3.3 Input and output of the web search phase 28
3.4 Input and output of the classification phase 29
3.5 Input and output of the code generation phase 32
3.6 Input and output of the code testing phase 34
3.7 Input and output of the code revision phase 36
3.8 Input and output of the sanity check phase 37
3.9 Input and output of the static validation phase 38
3.10 Input and output of the red team phase 40
3.11 Input and output of the debugger phase 42

5.1 Representation of the CVE dataset 56

List of Acronyms

Acronym	Expansion
AI	Artificial Intelligence
API	Application Programming Interface
CIA	Confidentiality, Integrity, Availability (security triad)
CLI	Command Line Interface
CNA	CVE Numbering Authority
CSRF	Cross-Site Request Forgery
CTF	Capture The Flag
CVE	Common Vulnerabilities and Exposures
CVSS	Common Vulnerability Scoring System
CWE	Common Weakness Enumeration
DL	Deep Learning
DNS	Domain Name System
DoS	Denial of Service
EPSS	Exploit Prediction Scoring System
FN	False Negative
FP	False Positive
HTTP	HyperText Transfer Protocol
IP	Internet Protocol
JSON	JavaScript Object Notation
LLM	Large Language Model
ML	Machine Learning
NIST	National Institute of Standards and Technology
NN	Neural Network
NVD	National Vulnerability Database
OS	Operating System
PoC	Proof of Concept
RCE	Remote Code Execution
SQL	Structured Query Language

Acronym	Expansion
SQLi	SQL Injection
SSH	Secure Shell
SSRF	Server-Side Request Forgery
SUID	Set User ID
TCP	Transmission Control Protocol
TN	True Negative
TP	True Positive
URL	Uniform Resource Locator
VM	Virtual Machine
XML	eXtensible Markup Language
XSS	Cross-Site Scripting
XXE	XML External Entity
YAML	YAML Ain't Markup Language

Chapter 1

Introduction

Nowadays, the increasing capabilities of Large Language Models (LLMs) in complex technical tasks have made them increasingly relevant to cybersecurity. In particular, one of the capabilities that has received significant attention is their ability to perform offensive security tasks, such as identifying and exploiting vulnerabilities in controlled environments. These capabilities are commonly evaluated through Capture The Flag (CTF) challenges, in which an agent must interact with a vulnerable system, exploit the intended weakness, and retrieve a hidden flag as evidence of success.

Such environments have applications beyond the evaluation of cybersecurity agents. They can be used, for example, in red teaming activities, for security education and CTF competitions, and for validating whether a software patch effectively removes a vulnerability. In all these scenarios, the environments need to be realistic, reproducible, and representative of vulnerabilities that are relevant at the time of the evaluation.

However, creating and maintaining such environments is still a largely manual process. The number of publicly disclosed vulnerabilities continues to grow, while the software versions and configurations required to reproduce them may become obsolete over time. As a consequence, manually constructed collections are expensive to maintain and difficult to scale, limiting the possibility of evaluating security agents against a large and continuously updated set of vulnerabilities.

Several approaches have been proposed to reduce this manual effort. Catalogue-based collections of vulnerable environments can provide ready-to-use playgrounds, but their coverage depends on the vulnerabilities that have previously been packaged and maintained. Source-driven approaches, such as CVE-GENIE[1], instead automate part of the process by starting from the source code of the affected application and reconstructing the vulnerable environment. This approach reduces

the manual effort required to build an environment, but requires the vulnerable application's source code to be available and usable.

A natural way to generalize the environment creation process is to use the CVE-ID of a vulnerability as the input to the system. A CVE identifies a specific publicly disclosed vulnerability, but a CVE description does not directly provide a runnable environment in which that vulnerability can be exploited. Generating such an environment requires the underlying technical information to be reconstructed: the affected software and version must be identified, the vulnerable service must be configured correctly, and the resulting environment must expose the conditions necessary for exploitation.

Vulnerable environment generation is a non-trivial task. An environment may build and run correctly while still failing to reproduce the intended vulnerability: indeed, an agent may identify an incorrect software version, select the wrong service, or overlook a configuration required to make the vulnerability reachable. The goal is therefore to produce a working container in which the intended vulnerability is actually exploitable, while having evidence that this is the case. The reasoning and iterative interaction this requires make the task a natural candidate for automation through agentic AI: LLMs can interpret technical descriptions, generate code and configuration files, execute commands, inspect the results, and iteratively revise their output, so an agent could start from a vulnerability description and construct the corresponding playground.

An independent mechanism is needed to check that in a generated environment the intended vulnerability is present and reachable. Validation can be approached statically or dynamically, with different trade-offs. Static analysis tools, such as Docker Scout[2], inspect the installed software and versions and check them against known vulnerable configurations: it is efficient, but it does not attempt an exploit and cannot, on its own, distinguish the presence of a vulnerable component from its actual exploitability. Dynamic validation instead interacts with the running environment and attempts to exploit the vulnerability, providing stronger evidence that it is reachable.

A dynamic approach is more informative, but it comes with its own challenges. The outcome of an exploitation attempt has to be both observable and trustworthy: the signal used to determine a successful outcome must reflect an effective exploit, and not, for example, an agent that uses its visibility over the environment to directly read the target. To determine the reliability of the validation, a ground truth is required: however, public Proofs of Concept (the natural reference) exist only for a fraction of CVEs and are not uniformly distributed across vulnerability types, release periods, and difficulty levels. A methodology that depends only on

public PoCs therefore cannot be applied uniformly across a large and diverse set of vulnerabilities.

These limitations motivate the research question addressed in this thesis: can an AI agent autonomously generate vulnerable playground environments starting only from the information identifying a CVE, while also providing evidence that the generated vulnerability is actually exploitable?

To investigate this question, building upon an existing baseline pipeline limited to environment generation and static validation [3], I designed and implemented a dual-agent framework, whose only vulnerability-specific input is the CVE-ID. I deliberately chose not to require the application's source code or a public PoC (Proof of Concept) as input. This choice allows the system to be applied to recent CVEs, for which neither a clean source-code snapshot nor a public PoC may be available.

The pipeline consists of two main agentic stages. The first is the *forger*, which receives the CVE-ID, reconstructs the affected service and its relevant configuration, and generates the corresponding containerized environment. During this process, the agent iteratively builds and tests the environment and injects a random flag in it. The second is the *evaluator*, a white-box red team agent that performs reconnaissance and attempts to exploit the generated environment in order to retrieve the flag.

The output of the pipeline is therefore not only a container that appears to contain a vulnerable software version. It is a container together with an independent demonstration that the intended vulnerability can actually be exploited. This separation also makes it possible to distinguish failures in environment generation from failures in autonomous exploitation.

The generation and evaluation stages are validated using different mechanisms. For a subset of the selected CVEs, a publicly available PoC was used to assess the vulnerability of the generated environment, while the generation process is organized in different steps, which allow to determine in which phase the generation failed.

I evaluated in a complete way the proposed approach on 150 CVEs selected across 25 distinct CWE categories, different release periods, and difficulty levels. The overall number of tested environments is 226. As a first validation method, Docker Scout was used to statically determine whether the generated environments contained software versions associated with the target vulnerabilities. For 94 CVEs for which a public PoC was available, the known exploit was executed against the generated environment, and generation was considered successful when the exploit retrieved the flag. To extend dynamic validation to the remaining CVEs, for which

no public PoC is available, the *evaluator* autonomously analyzed and attacked the generated environment. Rather than restricting it to the CVEs without a PoC, the *evaluator* was run against every generated environment, providing an independent cross-check on the outcome of the PoC-based validation.

The results show that the forger successfully produced a working, testable environment for 150 of the 226 CVEs considered (66.4%). Among these, the evaluator confirmed 38% as truly exploitable and, on the subset of 94 CVEs for which a public PoC was available, the evaluator's outcome agreed with the PoC's in 78.7% of cases. These results support the claim that the generated environments can reproduce the intended vulnerability. At the same time, the results show that static validation alone is not sufficient to assess the generated environments. Docker Scout fails to identify many environments that can be successfully exploited by the corresponding PoC, while the *evaluator* recovers most of these cases by dynamically retrieving the flag. Conversely, cases flagged only by Docker Scout are not confirmed by the corresponding PoCs, indicating that static detection can also produce false positives. Overall, these results provide evidence that AI agents can be used both to automate the construction of vulnerable playgrounds and to dynamically assess whether the generated vulnerabilities are actually exploitable.

Chapter 2

Background and Related Work

In this chapter, an overview of the main theoretical concepts discussed in this thesis is presented, in particular focusing on software security, containerization, and large language models.

2.1 Software Security: Definitions

2.1.1 Vulnerabilities

A vulnerability is a flaw in a system that can be exploited by an attacker to compromise the security of the system. A common definition of a vulnerability is given by the National Institute of Standards and Technology (NIST)[4]: *A weakness in an information system, system security procedures, internal controls, or implementation that could be exploited or triggered by a threat source*[5].
Another, more specific definition is given by NIST in their National Vulnerability Database (NVD)[6]: *A weakness in the computational logic (e.g., code) found in software and hardware components that, when exploited, results in a negative impact to confidentiality, integrity, or availability*[7].
These definitions rely on several related concepts that are useful to distinguish before discussing vulnerabilities in more detail.

- **Threat:** Any circumstance or event with the potential to adversely impact organizational operations, organizational assets, or individuals through an information system via unauthorized access, destruction, disclosure, modification of information, and/or denial of service[8].
- **Attack:** Any kind of malicious activity that attempts to collect, disrupt, deny,

degrade, or destroy information system resources or the information itself[9].

- **Weakness:** A condition in a software, firmware, hardware, or service component that, under certain circumstances, could contribute to the introduction of vulnerabilities[10].
- **Confidentiality:** Preserving authorized restrictions on information access and disclosure, including means for protecting personal privacy and proprietary information[11].
- **Integrity:** Guarding against improper information modification or destruction, and includes ensuring information non-repudiation and authenticity[12].
- **Availability:** Ensuring timely and reliable access to and use of information[13].

A vulnerability therefore represents a weakness that can lead to a compromise of one or more of the three main security properties: confidentiality, integrity, and availability. Identifying such weaknesses requires analyzing the target software from an appropriate level of visibility.

Discovering Vulnerabilities: White and Black Box Analysis

To look for vulnerabilities in a software product, two main families of approaches can be distinguished according to the level of visibility available to the analyst:

- **White-Box:** The entity has complete visibility over the source code and the resources used by the software;
- **Black-Box:** The entity is able to interact with the software without knowing implementation details, like an external attacker would do.

There exist intermediate solutions (*Grey-Box analysis*), in which for example only some implementation details are disclosed. The type of approach can be chosen according to different motivations.

The analysis of a software system may reveal one or more weaknesses, but a specific vulnerability must also be uniquely identified and publicly documented. This is the role of the Common Vulnerabilities and Exposures [14] (CVE) system.

2.1.2 Common Vulnerabilities and Exposures (CVE)

A CVE is a standard way of describing publicly disclosed cybersecurity vulnerabilities found in software. It is represented by a unique identifier (the CVE-ID) assigned by the CVE Numbering Authority (CNA) and it always includes a brief

description and at least a public reference.
CVE's main goal is to help organizations improve their security defenses. It does this by providing a list of software or firmware vulnerabilities and making it available as a free resource. Once a CVE identifies a risk, organizations must manage it. When a new vulnerability is discovered, the organization must inform the MITRE corporation[15]. MITRE is also in charge of maintaining the CVE list.
CVE-IDs are assigned to flaws that meet specific criteria:

- **Be independently fixable:** The flaw can be fixed independently of other bugs;
- **Be acknowledged by the affected vendor or documented:** The software vendor acknowledges the bug's existence and confirms that it negatively impacts security.
- **Affect only one codebase:** If a flaw impacts more than one product, it gets a separate CVE for each product.

While the CVE identifies a specific vulnerability, it may not describe the general category of weakness that caused it. For this reason, it is useful to rely on a CWE.

2.1.3 Common Weakness Enumeration (CWE)

CWE[16] is a community-developed list of common software and hardware weakness types that could have security ramifications. Each weakness is assigned a unique identifier (such as CWE-89, representing the "SQL Injection" weakness) together with a description, one or more examples, and information about possible mitigations to it. CWEs focus on the type of vulnerabilities rather than on specific instances of them, such as buffer overflows, SQL injection and cross-site scripting. Consequently, many different CVEs can be associated with the same CWE. To better distinguish among CVEs and CWEs, it is possible to think of a CVE as what specific issue exists in the software, while the CWE explains what type of error led to that issue, and how it is possible to prevent it.

2.1.4 Related Metrics: CVSS and EPSS

The Common Vulnerability Scoring System[17] (CVSS) is a free and open industry standard for assessing the severity of security vulnerabilities. It is represented by a *Base* score, which is a number between 0 (low severity) and 10 (high severity), derived from different metrics, such as the impact over the CIA triad, the attack vector and the complexity. This value is then modified by an environmental factor and a temporal factor: the former takes into account the characteristics of a vulnerability that are unique to a user's environment, while the latter considers

the characteristics of a vulnerability that changes over time.
The Exploit Prediction Scoring System[18] (EPSS) is used to estimate the likelihood that a vulnerability will be exploited in the wild, through a data-driven estimation. It associates each CVE with a score between 0 and 1, representing the probability of an exploit. This value can be used to better prioritize the vulnerability remediation efforts, as it combines real-world exploit data with current threat intelligence.

These metrics describe the characteristics and expected relevance of a vulnerability, but they do not demonstrate that the vulnerability can actually be exploited. To address this distinction, it is necessary to consider the exploit itself and the conditions required for its successful execution.

2.1.5 Exploit and Proof of Concept

According to Cisco, an exploit is a program, or piece of code, designed to find and take advantage of a security flaw or vulnerability in an application or computer system, typically for malicious purposes such as installing malware[19]. An exploit is not malware itself, but rather it is a method used by cybercriminals to use the vulnerability for their objective, which may be delivering a malware, escalating the privileges, etc.
On the other hand, a Proof of Concept (PoC), in this context, is a non-harmful exploit that demonstrates the existence of a vulnerability, how it can be exploited, and what is its potential impact.

2.1.6 Software Patches

A software patch is an update provided by the software vendor to correct a defect or security vulnerability in an existing software component. A patch is the immediate solution to an identified problem or vulnerability that is provided to users by the software vendor. Vulnerabilities can be categorized, according to the existence of a patch, as:

- **Patched:** a patch is available for this vulnerability;
- **Unpatched:** a patch is not available, but the vulnerability is known and documented;
- **Zero-Day:** the vulnerability is completely unknown to the software vendor;

In fact, vulnerable playgrounds can also be used to study patching scenarios, for example by comparing the behavior of a vulnerable environment with that of a patched one.

2.1.7 Attack Vectors

An attack vector is a pathway or method through which hackers gain unauthorized access to target systems to commit a cyberattack. Some examples of attack vectors are *phishing*, *Denial of Service (DoS)* and *Insider Threats*.
CVSS classifies attack vectors in four main categories:

- **Network:** The vulnerable component is bound to the network stack, and the attacker can exploit the flaw remotely;
- **Local:** The component is not bound to the network, and the exploit requires the attacker to have physical or logical terminal access, for example using a shell;
- **Adjacent:** The attacker must be on the same shared physical or logical network (such as a shared local IP subnet) to reach the vulnerable target;
- **Physical:** The exploit demands direct physical manipulation, interaction, or theft of the hardware device itself;

In the context of this thesis, the attack vector is used both to characterize a vulnerability and to determine how the generated environment should be exposed to the *evaluator*. For example, vulnerabilities classified as Local require a form of local access to the target system.

2.1.8 Kill Chain

While the attack vector describes how an attacker initially reaches the vulnerable component, a successful attack may require multiple consecutive stages. The Cyber Kill Chain[20] provides a high-level model for describing them. Typically, the kill chain is divided into seven stages:

1. **Reconnaissance:** Attackers retrieve information about their target to find potential entry points and identify possible vulnerabilities;
2. **Weaponization:** The information gathered in the previous step is used to build malware or payloads able to exploit the weaknesses found;
3. **Delivery:** Attempt to infect the target device by delivering the malware. It is possible to perform this step using social engineering, phishing or exploiting vulnerabilities;
4. **Exploitation:** The malware delivered is used to exploit some weaknesses in the target device. It usually performs a *lateral movement* to propagate the infection in the internal network or across other devices, identifying other vulnerabilities;

5. **Installation:** In this phase, attackers use techniques such as backdoors, Trojan horses or access token manipulation to gain more control over the infected network, in order to escalate privileges and change permissions;

6. **Command and Control:** Attackers establish a channel to remotely guide and monitor their operations in the infected device. They can use obfuscation techniques to cover their tracks;

7. **Actions on Objectives:** This final phase consists in the execution of the attack's primary objective, for example the encryption of the infected devices (in order to ask for a ransom) or the exfiltration of sensitive data;

The Cyber Kill Chain is used in this context as a high-level characterization of the exploitation process associated with a CVE. The number and nature of the attack stages required to successfully exploit a vulnerability are later used as an indicator of its complexity.

2.1.9 Selected Vulnerability Classes

This section provides a summary of common classes of vulnerabilities. These categories are used in the methodology to describe and classify the vulnerabilities considered in the experiments.

Remote Code Execution

Remote Code Execution (RCE) is a class of vulnerabilities in which an attacker is able to execute arbitrary commands or code on a target system, typically without being authenticated. These vulnerabilities are typically triggered when attacker-controlled input is passed to a system shell or a function that executes code (CWE-78, OS Command Injection; CWE-94, Code Injection).

Insecure Deserialization

Insecure deserialization (CWE-502) is a class of vulnerabilities affecting many programming languages and frameworks that provide mechanisms to reconstruct objects from a serialized byte stream. If this stream originates from an untrusted source, an attacker can craft a malicious payload that, once deserialized, triggers arbitrary code execution through the target application.

SQL Injection

SQL Injection (SQLi, CWE-89) is possible when user input is concatenated into a SQL query without proper sanitization or parameterization, allowing an attacker

to change the logic of the query. Exploitation techniques range from *error-based* injection (where database error messages leak data), to *UNION-based* injection (where an attacker appends a crafted `SELECT` statement to read table contents), to *blind* injection, in which no data is directly returned and the attacker must infer information. A successful exploitation can lead to unauthorized read or write access to the underlying database.

XML External Entities (XXE)

XML External Entity (XXE, CWE-611) vulnerabilities affect applications that parse XML input using a parser configured to resolve external entities. An attacker can define a custom entity referencing a local file or a URL: when the parser resolves it, the referenced content is included in the application's response, allowing arbitrary local file disclosure or the exfiltration of blind data.

Path Traversal

Path Traversal (also referred to as directory traversal, CWE-22) allows an attacker to access files or directories located outside the folder intended to be exposed by the application, typically by manipulating a file path parameter with sequences such as `../`. Exploiting this vulnerability can lead to arbitrary file access.

Server Side Request Forgery (SSRF)

Server Side Request Forgery (SSRF, CWE-918) affects applications that can be induced to issue network requests to a destination chosen by the attacker. It is particularly dangerous in containerized and cloud environments, where the vulnerable server often has network access to internal services that are not directly reachable from the outside. An attacker can therefore use the vulnerable application as a network proxy to reach and interact with internal resources.

Cross Site Scripting (XSS)

Cross Site Scripting (XSS, CWE-79) allows an attacker to inject attacker-controlled script code into a web page that is subsequently rendered in another user's browser, in the security context of the vulnerable site. Exploitation can lead to session hijacking, credential theft, or other actions the victim is authorized to perform in the application.

Cross Site Request Forgery (CSRF)

Cross Site Request Forgery (CSRF, CWE-352) is a web security vulnerability that allows an attacker to induce users to perform actions that they do not intend

to perform. An attacker can craft a malicious page that, when visited by an authenticated victim, silently triggers a state-changing request to the vulnerable application, which cannot distinguish it from a legitimate action performed by the user.

Authentication Bypass

Authentication Bypass (CWE-287) is a kind of flaw that allows an attacker to access data that should be reachable only after a successful authentication step. This can result from many different causes, such as insecure default credentials or logic errors in the authentication flow. The direct consequence is that the attacker gains access to a privilege level they should not have.

Privilege Escalation

Privilege Escalation (CWE-269) is a class of vulnerabilities that allow an attacker to obtain a higher level of privilege than intended inside the system. Common root causes include improperly configured file permissions, exploitable SUID binaries, misconfigured `sudo` rules, or writable scheduled tasks (cron jobs) that execute with elevated privileges. Unlike most of the other classes discussed in this section, privilege escalation is generally a *local* vulnerability, in the sense that it requires the attacker to have already some form of access to the target system.

Information Disclosure

Information Disclosure (CWE-200) is a broad class of vulnerabilities in which an application unintentionally exposes sensitive information (such as source code, credentials, internal file paths, or debugging output) to an unauthorized party. Although often considered less severe than vulnerabilities that lead directly to code execution, information disclosure frequently provides an attacker with the reconnaissance needed to plan and execute a more impactful attack.

2.2 Virtualization and Containerization

To be effective, a vulnerable environment should provide the correct conditions to exploit a particular vulnerability. Indeed, automatically building an environment in which a CVE is actually exploitable requires both the vulnerable software and the environment in which it runs to be correctly configured. This raises the question of how such environments can be built in a reproducible, isolated, and automated way.

2.2.1 Virtualization

Virtualization is a technique in which software is used to create an abstraction layer over the hardware. It allows hardware elements (such as processors, memory, storage, etc.) to be divided into multiple virtual computers, commonly called Virtual Machines (VMs). Each VM runs its own Operating System (OS) and behaves like an independent computer, even though it is running on just a portion of the actual underlying computer hardware. Multiple VMs can run on the same hardware. Virtualization provides several useful properties for isolated environments, including the ability to create independent computing instances, reproduce existing system configurations, and isolate workloads through the hypervisor. However, each virtual machine requires a complete guest operating system, which introduces a non-negligible resource and deployment overhead.
Although virtual machines provide strong isolation, requiring a complete guest operating system for each instance can be unnecessarily expensive when many short-lived environments must be created. This motivates lighter isolation mechanisms based on operating-system virtualization.

2.2.2 Lightweight Virtualization and Containers

Lightweight virtualization was introduced in order to create a system that has the benefits of virtualization but that consumes fewer resources. It is appropriate to use this technique when an isolated environment that is quick to deploy, scale and migrate is required, or when it is not possible to accept the overhead introduced with classical virtualization.

Containers are the main example of lightweight virtualization. Containers provide a lighter form of isolation by packaging an application together with its dependencies while sharing the host operating system kernel. Instead of emulating a complete guest operating system, containers isolate groups of processes using operating-system mechanisms for resource and namespace isolation. As a result, containers can be created and started with significantly lower overhead than virtual machines, making them well suited for reproducible environments that must be deployed and destroyed frequently.

2.2.3 Docker

Docker [21] is the best-known available tool to create and run Linux containers. It is an open-source platform that allows developers to create, execute and manage containers. It became so popular because it is more user-friendly than its alternatives, requiring only simple commands and exposing useful application programming

interfaces (API). These capabilities make Docker suitable for implementing the reproducible vulnerable playgrounds considered in this thesis.

Docker Engine and Docker Daemon

The Docker Engine is a client-server application that includes the Docker Daemon, an API to interact with the daemon, and a command line interface. The Docker Daemon is the service in charge of generating and managing the Docker images based on the client's commands.

Docker Images

Docker images contain the filesystem of the application, together with all the libraries, dependencies and tools that are required by the application to correctly run. When a Docker image is executed, it generates an *instance* of it. From a single image it is possible to generate many different instances. Docker images are composed of layers, where each layer corresponds to a version of the image. Any time a developer modifies the image, a new layer is created on top of the others, representing the current version of the image. Older layers are preserved, in order to allow a roll-back procedure.
The contents and build process of an image are specified through a Dockerfile.

Dockerfile

A Dockerfile is a text file containing the instructions required to build a Docker image. It therefore provides a declarative description of the software, dependencies, files, and configuration that should be included in the generated environment.

Docker Compose

Developers can use Docker Compose to manage multi-container applications, in which all the containers run on the same host. It is possible to use a single YAML file (`docker-compose.yml`) to declare the services that are included in the application. Then, it is possible to use that file to implement and run all the containers with a single command. Installing the vulnerable software is, however, not sufficient to reproduce an exploitable environment. The vulnerable component must also be reachable through an appropriate attack surface, which may depend on the network configuration of the environment.

2.2.4 Container Networking

Container networking refers to the ability of containers to connect to and communicate with each other, and with non-Docker network services. Containers

have networking enabled by default, and they can make outgoing connections. A container has no information about what kind of network it is attached to, or whether its network peers are also Docker containers. A container only sees a network interface with an IP address, a gateway, a routing table, DNS services, and other networking details. Docker Engine by default has a single built-in network called the *default bridge* network. If not differently specified, when a container is run it is connected to the default bridge. Containers attached to the default bridge have access to network services outside the Docker host. It is possible to create user-defined networks, and connect groups of containers to the same network. Once connected to a user-defined network, containers can communicate with each other using container IP addresses or container names. It can be useful to separate groups of containers that should have full access to each other. Indeed, with the default configuration, containers attached to the default bridge network have unrestricted network access to each other using container IP addresses.

2.3 LLMs and AI Agents

Constructing such environments manually requires understanding the vulnerability, identifying the affected software, determining its configuration and dependencies, and finally producing a working deployment. These characteristics make the task a candidate for automation through systems capable of reasoning over technical descriptions and interacting with external tools.

2.3.1 Artificial Intelligence and Machine Learning

Artificial Intelligence (AI) is a technology that enables computers to emulate human thought and behavior. AI's main goal is to create advanced systems that are able to emulate human intelligence, functioning and making decisions autonomously. AI systems are able to learn from new information and experience.

Machine Learning (ML) is a subset of AI, in which the computer learns from previous experience or given data. Machine learning algorithms' performance is strongly dependent on the quality of the data used to train them: if such data is of a poor quality, even the best algorithm will have poor performance. Those algorithms are used to create models that are able to make predictions and decisions based on the training data.

Neural Networks (NN) are a kind of machine learning algorithm, which is inspired by the behavior of the human brain. It is composed of nodes (the *neurons*), that are organized in layers connected through *synapses* to propagate information between them. These algorithms are used to model complex, non-linear problems,

which are difficult to be effectively represented with classical machine learning techniques.

Deep Learning (DL) is a subset of machine learning that relies on multi-layered neural networks. They include an *input layer*, at least three (but usually many more) *hidden layers*, and an *output layer*. Increasing the number of hidden layers can enable the neural network to understand more complex patterns and relationships among the training data. The complexity of these algorithms makes it possible to process a very large amount of data, which is something unfeasible with classical algorithms. There exist different Deep Learning architectures based on the task for which the model is created.

2.3.2 Large Language Models and Prompts

A Large Language Model (LLM) is a type of deep learning model, trained on very large collections of text to predict and generate natural language.

Tokenization

Before being processed by the model, text must be converted into a numerical form. This is done through *tokenization*: the input text is split into smaller units called *tokens*, which can correspond to words, subwords, or even single characters, depending on the tokenization algorithm used. Each token is then mapped to a numerical vector, called an *embedding*, before being fed into the model.

The Transformer

LLMs are typically based on the **Transformer** architecture, first introduced in 2017 by *Vaswani et al.* in the paper "*Attention Is All You Need*" [22]. The central component of this architecture is the *self-attention* mechanism, which allows the model to understand the relevance of every other token in the input when processing a given token, regardless of their distance in the input sequence. This is a key improvement over previous sequential architectures (such as recurrent neural networks), which processed text one word at a time: those models had difficulty in retaining information over long sequences. Because self-attention computes these relationships in parallel rather than sequentially, Transformers can be trained much more efficiently on large amounts of data. This is indeed one of the main reasons behind the increasing diffusion and improvement of LLMs.

Pre-Training

LLMs acquire their general language understanding through *pre-training*, a self-supervised process in which the model is trained on very large text corpora to predict the next token in a sequence, given the preceding ones. Since this task does not require labelled data, it is possible to pre-train LLMs on a huge amount of text, either collected from books, the Internet or other sources.
It is important to notice that pre-training is only performed up to a certain point in time, the *knowledge cutoff*: any event that happened after this date will not be known by the LLM, unless it is explicitly provided as part of the input at inference time. This distinction is particularly relevant when using LLMs for security-related tasks: a model may appear to solve a problem correctly simply because it has memorized information about it during training, rather than because it is able to reason about a new case.

Fine-Tuning

After pre-training, a model is typically adapted to a more specific domain through *fine-tuning*, in which it is further trained on a smaller, labelled dataset that is more task-specific. In this way, starting from the same pre-trained model, it is possible to create different fine-tuned models for different tasks. Pre-training and fine-tuning enable reaching the same performance as models trained on impractically large labelled datasets using a much smaller amount of labelled samples.

Few-Shot and Zero-Shot Learners

Fine-tuning still requires a dataset of labelled data: OpenAI discovered that better pre-training unlocked hidden properties of the model [23]. If pre-trained on a sufficiently large general dataset, LLMs can be *Few-Shot learners*: this means that model can perform a new task after seeing just a few examples of that task in the prompt, without additional fine-tuning.
In 2022, Google discovered that fine-tuned models are *Zero-Shot learners* [24], meaning that they are able to perform well on tasks that were never seen during training. To be zero-shot learners, the models should be fine-tuned for as many tasks as possible.

Prompts

A prompt is the textual input given to an LLM. Prompts are typically divided into a *system prompt*, which defines the model's role, behavior, and constraints for the entire interaction, and a *user prompt*, which contains the specific request. Since LLM outputs are, by default, unstructured natural language, techniques for *structured*

output generation are often used to constrain the model's response to a predefined schema (e.g., a fixed set of fields), making it easier to process programmatically. A well-known limitation of LLMs is *hallucination*: the generation of content that is plausible in form but factually incorrect or unsupported by the given context, without any explicit signal that the output may be unreliable.

2.3.3 LLMs and AI Agents

While an LLM in isolation can only generate text, an *AI agent* extends this capability by allowing the model to perceive an external environment and act upon it through a set of predefined *tools* (such as executing a shell command, reading or writing a file). This is usually achieved through *tool use* (or *function calling*), in which the LLM decides which tool to invoke and with which parameters, based on the current context. Then the LLM observes the result and uses it to decide the next action. This iterative cycle of reasoning and acting allows an agent to complete complex tasks requiring multiple steps by interacting with a real system, rather than simply producing a single textual answer.

2.3.4 Multi-Agent Systems

A complex task can be decomposed across multiple specialized agents rather than being assigned entirely to a single agent. In a multi-agent system, each agent is responsible for a distinct sub-task and agents can cooperate by exchanging information or evaluating one another's results. This separation is useful when the task includes both generation and evaluation, since the agent responsible for producing a solution can be kept distinct from the agent responsible for assessing or attacking it. In this thesis, this design is reflected in the separation between the *forger*, which constructs the vulnerable environment, and the *evaluator*, which independently attempts to exploit it.

2.4 Security Testing and Validation Methodology

The ability to generate an environment is not sufficient on its own. A generated container may build successfully while still failing to reproduce the intended vulnerability. Therefore, an independent validation mechanism is required to determine whether the resulting environment is correctly built and also actually exploitable.

2.4.1 Static and Dynamic Analysis

Software analysis techniques for identifying vulnerabilities are broadly divided into two categories. *Static analysis* examines a system without executing it (for example by inspecting source code, configuration files, or the versions of installed packages) to infer whether known vulnerabilities may be present. *Dynamic analysis*, on the other hand, evaluates a system while it is running, typically by interacting with it and observing its actual behavior. Static analysis is generally faster and cheaper to perform, but it can produce *false positives* (i.e. it may flag a vulnerability that cannot actually be triggered) and *false negatives* (missing an exploitable weakness), since it does not verify whether the flaw is actually reachable and exploitable in practice. Static and dynamic analysis therefore provide different types of evidence about the presence and exploitability of a vulnerability. To turn this evidence into a binary decision about whether a test has succeeded, it is possible to employ a testing oracle.

2.4.2 Testing Oracles

In software testing, an *oracle* is a mechanism used to determine whether the outcome of a test is correct. In the context of vulnerability assessment, an oracle decides whether a given system is actually vulnerable. A *static oracle* bases this decision on static analysis alone (e.g., matching installed package versions against a vulnerability database), while a *dynamic oracle* bases it on the observed outcome of an actual exploitation attempt, providing stronger evidence at the cost of requiring the system to be deployed and actively probed. In this thesis, Docker Scout[2] can be regarded as a static oracle, whereas successful exploitation followed by flag retrieval provides a dynamic oracle.

2.4.3 Ground Truth and Evaluation Metrics

In an evaluation setting, the *ground truth* is the reference outcome against which a system's prediction is compared. When evaluating a binary decision, such as whether an environment is vulnerable or not, predictions can be classified as true positive (TP), true negative (TN), false positive (FP), or false negative (FN). This classification, commonly represented through a confusion matrix, allows the reliability of a validation mechanism to be quantified when a ground truth outcome is available.

In the context of vulnerability exploitation, obtaining such a reference outcome requires an observable condition that distinguishes a successful exploit from an unsuccessful one. A CTF-style flag provides a natural mechanism for this purpose.

2.4.4 Capture The Flag (CTF)

Capture The Flag (CTF) is a competitive format widely used in cybersecurity training and evaluation, in which participants must exploit a vulnerability to retrieve a hidden evidence called *flag*. In this setting, the flag is placed so that it can be retrieved only after a successful exploitation of the target vulnerability. Its capture therefore provides an observable, binary signal of exploitation success. This property makes the CTF format particularly suitable for the dynamic validation performed in this work, where the *evaluator* must retrieve the flag from the generated environment.

2.4.5 Penetration Testing, Red Teaming and Blue Teaming

Penetration testing is the practice of simulating a cyberattack against a system, under authorized and controlled conditions, in order to identify exploitable weaknesses before real attackers can find them. *Red teaming* extends this concept to a broader, often more adversarial and less constrained simulation of a real attacker's behavior, evaluating not only technical vulnerabilities but also an organization's overall detection and response capabilities. In contrast, *blue teaming* refers to the defensive counterpart: the activity of detecting, responding to, and mitigating an ongoing or simulated attack. In this thesis, the term *red team agent* refers to an AI agent tasked with autonomously performing the offensive role typical of red teaming, in order to dynamically verify the exploitability of a given vulnerability.

2.5 Related Work

This thesis can be positioned at the intersection of three research areas: the construction of reproducible vulnerable environments, the evaluation of LLM-based cybersecurity agents, and the automation of vulnerability exploitation and validation. Existing approaches address different parts of this problem, but they assume in general that at least one of these components is already available. The following sections review the most relevant approaches and position the proposed system with respect to them.

2.5.1 Curated Vulnerable Environments

A first approach to obtain vulnerable environments consists in providing a curated collection of pre-built instances. Vulhub[25] is a concrete example of this approach: it is an open-source repository of vulnerable environments implemented using Docker and Docker Compose. Each environment in the collection is associated

with a specific vulnerability and can be provided with some documentation that describes its reproduction and exploitation.

The main limitation of this approach is that the environments must be created and maintained by humans. Since the number of publicly disclosed vulnerabilities increases, extending the collection requires additional engineering effort. Maintaining an up-to-date collection becomes increasingly difficult also because of software versions, dependencies, and vulnerable configurations change over time. Vulhub provides therefore an important source of vulnerable environments, but it does not address the problem of automatically generating them.

2.5.2 Automated Vulnerable-Environment Generation

A more direct approach to reduce the manual effort is to automate the construction of vulnerable environments. CVE-GENIE[1] addresses this problem by using an agentic pipeline to reproduce vulnerabilities starting from CVE-related information. The system extracts the relevant vulnerability information, identifies the vulnerable software version, constructs the environment, and can subsequently build and verify the resulting instance. The project also supports automated exploitation and verification of the reproduced vulnerability.

CVE-GENIE[1] is closely related to the approach proposed in this thesis, since both systems aim to automate the transition from vulnerability information to an exploitable environment. An important difference between the two approaches is represented by the architectural requirements and assumptions regarding the available software artifacts. While CVE-GENIE does not strictly require a public proof of concept (PoC), as it can successfully synthesize exploits by analyzing *patch diffs*, it requires the possibility to access the complete source code of the affected application. It requires it in order to explore, compile, and configure the vulnerable environment. Instead, the pipeline proposed here operates under fewer requirements: it uses the CVE-ID as the primary input but does not require access to the application's source code or any public PoC. In this case, the proposed approach is provided with less knowledge of the vulnerability, but these constraints make it suitable for vulnerabilities in closed-source software or for newly disclosed CVEs. In particular, source snapshots, reproduction instructions, or public PoCs related to the newest CVEs may not yet be available, even though the vulnerability itself is already known. Automating environment generation under these weaker assumptions addresses a different and more general problem than reproducing a vulnerability from complete and accessible software source repositories.

2.5.3 Benchmarks for LLM-Based Cybersecurity Agents

Other research focuses on evaluating the capabilities of LLM-based cybersecurity agents. Several recent benchmarks use interactive CTF or penetration testing environments to measure whether agents can perform reconnaissance, plan multi-step attacks, use security tools, and successfully exploit vulnerable systems.

NYU CTF Bench[26] provides a collection of challenging, real-world CTF tasks specifically designed to evaluate LLM agents on interactive offensive-security problems and complex automated task planning. Similarly, CyBench[27] provides a framework for specifying and evaluating cybersecurity tasks for language-model agents, with the goal of measuring their ability to identify vulnerabilities and execute attacks across a collection of cybersecurity challenges.

These benchmarks are important because they provide standardized environments in which the offensive capabilities of LLM agents can be measured. From the perspective of the present thesis, they generally treat the construction of the benchmark environments as a prerequisite rather than as part of the benchmark generation problem. Their usefulness therefore depends on the availability, quality, and coverage of the underlying challenge collection, which is typically manually curated and static.

More recent work has attempted to reduce the gap between abstracted CTF tasks and real-world vulnerabilities. CVE-Bench[28], for example, is designed to evaluate AI agents on the exploitation of real-world web application vulnerabilities identified by CVEs. The authors explicitly identify the reproducibility and systematic evaluation of real vulnerabilities as challenges that must be addressed when constructing such a benchmark.

2.5.4 Autonomous Penetration Testing

A complementary research direction investigates whether LLMs can autonomously perform penetration testing against existing targets. AutoPentest[29] and PentestGPT[30] are examples of this approach, that combine an LLM with specialized modules for reasoning, tool use, and interpretation of penetration-testing outputs. The resulting system is then evaluated on real-world penetration-testing targets and CTF challenges.

Such systems demonstrate the feasibility of using LLM-based agents as autonomous attackers, and are closely related to the *evaluator* component of the proposed pipeline. However, their task typically starts from an already deployed target, so environment construction and vulnerability reproduction are outside the main scope of the attacker.

The generation and exploitation problems are therefore often studied independently: one line of work provides or constructs vulnerable environments, while another develops agents capable of attacking them. The proposed system combines

these two capabilities by using one agent to construct the environment and a separate agent to independently attempt its exploitation.

2.5.5 Positioning of the Proposed Work

The approaches discussed above can be summarized along three dimensions: how the vulnerable environment is obtained, what information is required to construct it, and whether its vulnerability is dynamically validated. To properly contextualize the architectural evolution of this research, the comparison also includes the baseline pipeline [3] from which this project originated. While that preliminary work successfully automated environment generation and static validation, it lacked any autonomous and dynamic exploitation capabilities.

Table 2.1: Comparison between representative related approaches and the proposed work.

Approach	Env generation	Input	Dynamic validation
Vulhub	Manual / curated	Vuln-specific artifacts	No
CVE-GENIE	Automated	Src code + CVE info	Yes
CTF benchmarks	Pre-existing	Prepared challenge	Yes
CVE-Bench	Benchmark construction	Real-world CVEs	Yes
PentestGPT	No	Existing target	Yes
Baseline Pipeline [3]	Yes	CVE-ID	No
This thesis	**Automated**	**CVE-ID**	**Yes**

The main contribution of this thesis is therefore to introduce vulnerable environments, automated security agents and dynamic exploitation in combination under a weaker input assumption. Given a CVE-ID, the *forger* agent must reconstruct a working and exploitable environment without relying on the application's source code or on a known exploit. The resulting environment is then validated by the *evaluator*, while static validation and public PoCs are used as complementary reference mechanisms. This combination addresses the gap between the automated generation of vulnerable playgrounds and their dynamic validation, enabling the construction of larger and more up-to-date evaluation environments for cybersecurity agents.

Chapter 3

Methodology

In this chapter, I describe the main agentic workflow implemented to create vulnerable environments. This chapter first focuses on the environment generation phases, then it describes the static and dynamic validation steps. Finally, I present a description of the procedure used to retrieve a public Proof of Concept (PoC) and adapt it to run on the local environment.

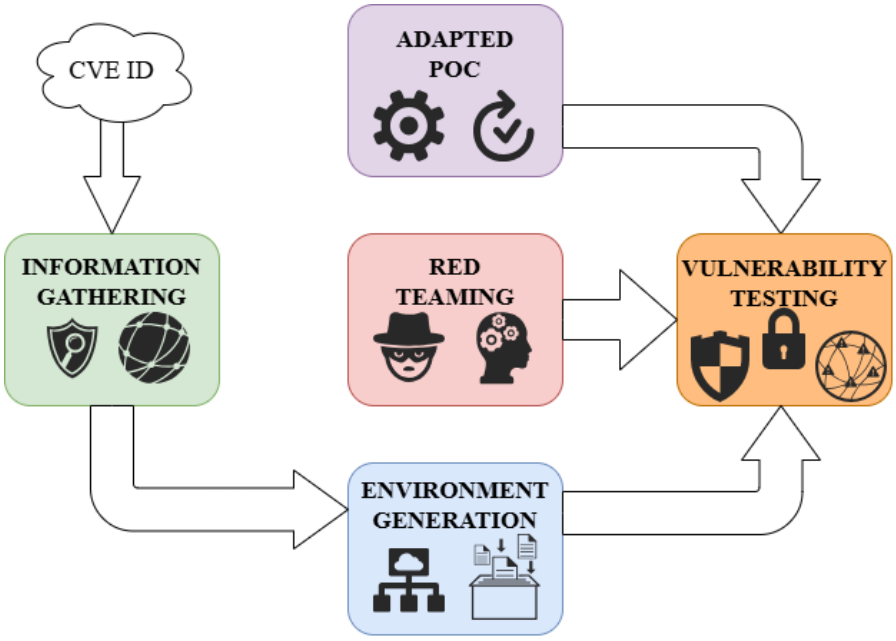


Figure 3.1: The methodology adopted to create and validate vulnerable environments.

Overall, the proposed methodology consists of different phases, described in Figure 3.1. The *forger*'s execution can be decomposed in the green block and the

blue block, while the *evaluator*'s in the red block. In parallel, the purple block represents the adaptation of public PoCs, performed in order to let them run on the generated environment. This step is executed outside the pipeline, since the generated environment must be already available. Finally, the orange block portrays the three different validation metrics adopted, which are summarized in the Table 3.1:

Metric	**Validation**	**LLM Involved**	**LLM interaction**
Evaluator-based	Dynamic	Yes	Discovery of the flaw Synthesis of exploit
Docker Scout	Static	No	None
PoC-based	Dynamic	Yes	Adapt an existing PoC

Table 3.1: Summary of the validation strategies adopted in this thesis

3.1 Pipeline Overview

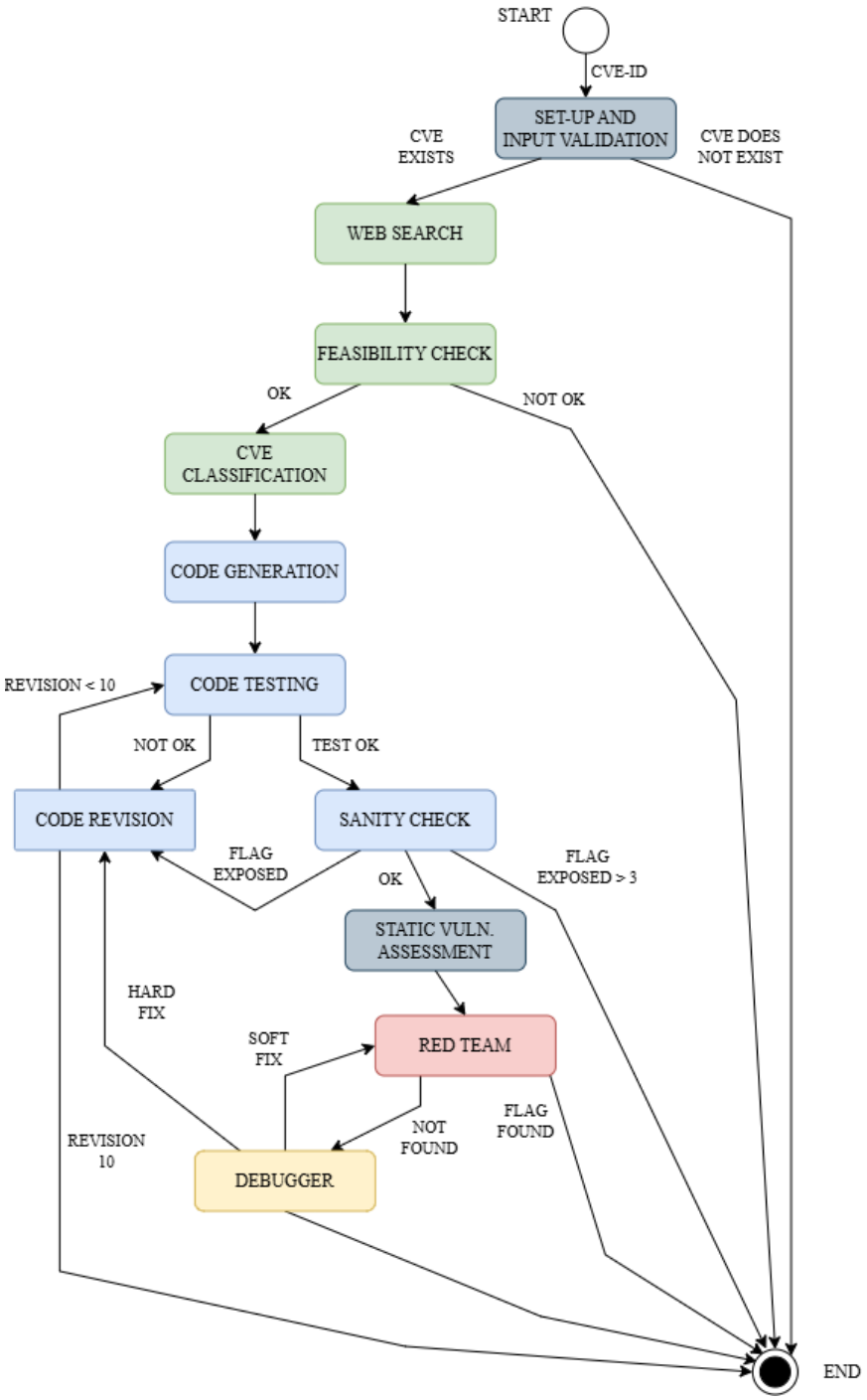


Figure 3.2: The pipeline used to generate vulnerable environments

The pipeline consists of four different stages, represented in Figure 3.2 using different colors:

1. Information Gathering (green): the agent searches the web, looking for information related to the input CVE-ID;

2. Environment Generation (blue): the information retrieved is used to generate the environment, that is tested to ensure it is running correctly;

3. Validation (red): the environment built is statically validated by Docker Scout and dynamically validated by the red team agent. The static validation is performed without an agentic interaction, therefore is represented in grey and not in red;

4. Debugging (yellow): if the dynamic evaluation fails, the agent uses the execution logs to understand what was the issue (which can be an exploit failed, a bad configuration, etc.) and attempts to fix it.

Only the dynamic part of the third stage is handled by the *evaluator* agent, while the other stages are executed by the *forger* agent.
Each step in the pipeline is represented as a LangGraph[31] node. The only exception is represented by the *RedTeam* node, which is part of the main graph but is used to invoke the *evaluator* as an Opencode[32] agent in an external process. I will discuss the implementation details more in depth in Chapter 4.

In the following sections, I describe the main features of each pipeline node more in detail. The agentic workflow is preceded by a setup phase, in which the LLM model is chosen and the state variables are initialized. Then the input is validated: the only required input is the CVE-ID of the vulnerability I want to represent in the vulnerable environment. For this purpose, the Mitre CVE Database is queried through their API, in order to check if the CVE-ID exists.
If this check is satisfied (i.e. the request returns a status code equal to 200), the pipeline can start.

3.2 Building the Environment: the Forger Agent

The *forger* agent is in charge of generating the vulnerable Docker environment, starting from the input CVE-ID.
To do so, its workflow goes through the steps presented in the following sections.

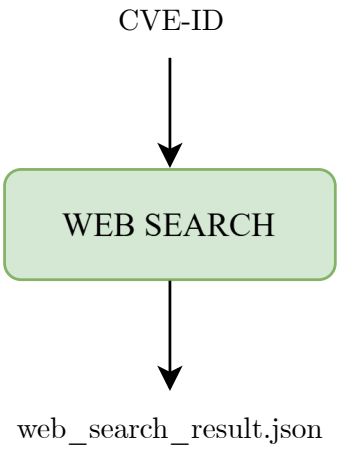


Figure 3.3: Input and output of the web search phase

3.2.1 Web Search and Platform Feasibility Check

The first task of the agent is to search the web to find information about the provided CVE-ID. It searches up to 10 different web pages using the Google API, from which are considered the five most relevant results containing valid content. The content fetched is summarized by the agent, in order to extract the most relevant parts of them. Then, the agent uses the NIST's NVD API and the GitHub Security Advisory to extract other useful technical information. Finally, a summary is created using all the sources retrieved and stored in a JSON file. That summary is then used by the agent to extract and store in a structured way the services affected by the vulnerability. In this summary, the services related to the vulnerability are split in two categories:

- **Hard Services:** this first category represents the services that are actually affected by the vulnerability;
- **Soft Services:** the second category instead represents the services that are required to create a working environment hosting the vulnerable service.

The information about the services required to build the vulnerable environment is first used to determine whether the vulnerability is compatible with the constraints adopted. Two different checks are performed: first, the services are deterministically checked with some regular expressions, so that services known to not be representable in Docker on Linux are discovered. Then, the agent checks the list of the hard services specifically, determining if they are representable in a Linux-based Docker container. If any service (hard or soft) fails the first check, or if a hard service fails the second, the pipeline is blocked.

3.2.2 Vulnerability Classification and Flag Placement

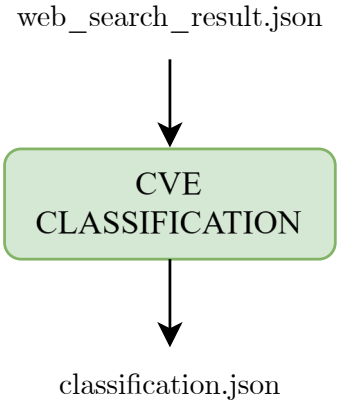


Figure 3.4: Input and output of the classification phase

In this phase, the agent uses its internal knowledge and the summary created after the web search to produce a classification report, stored in a JSON file. The Pydantic[33] class used to define the structure of the file is reported in the Appendix (Section A.3). The agent follows these steps:

- Identifies the root cause of the vulnerability, determining its CWE, the attack vector, and the vulnerable component (such as an HTTP endpoint or a CLI parameter);
- Describes possible exploit mechanics, in terms of: the input controlled by the attacker, the direct consequences of an exploit (such as arbitrary command execution), and how many steps are required for the exploit (the "kill chain");
- Determines the execution context user, that is the entity under which the attacker first obtains command execution during the exploit;
- Formulates a possible sequence of operations that an attacker should perform to exploit the vulnerability.

Then, the agent determines where a CTF-style flag should be placed inside the environment, based on the classification report. The flag is always represented in this phase by the string `FLAG{verified_CVE-X-Y}`, where CVE-X-Y is exactly the input CVE-ID. In this context, the agent must also define the user that should own the flag file and the permissions that should be set.

An important factor in the creation of the flag is represented by the vulnerability class the agent has associated to the CVE. The different classes handled by the pipeline are reported in the Appendix (Section A.1): the agent must choose from

this list the most appropriate one. It is important to underline that the meaning of *vulnerability class* here is related to some macro-categories of vulnerabilities, which have some common characteristics that can be used to determine how to handle the flag placement. Different strategies to determine how the flag should be created are implemented for each of the eleven defined class (listed in Section A.1). Here are reported some relevant examples.

Command Injection: The flag should be placed at `/etc/flag.txt`, owned by the user identified in the previous step, with permissions 400. In this way, to read the flag the agent must produce a working exploit that is able to execute arbitrary commands at low privileges.

Path Traversal: The flag should be placed outside the web root (or, in general, outside the directory that is first accessed when interacting with the system), for example at `/etc/flag.txt`, but owned by root, with 444 permissions. In this way, the flag is world-readable but not directly downloadable.

SQL Injection: The flag should be stored in a database record. A table *flag_table* is created in the database of the application, containing the column *flag*. To read the flag inside that table, the attacker must be able to execute arbitrary SQL commands.

Server Side Request Forgery: The flag should be placed in a hidden container, reachable only by triggering the vulnerability in the exposed container. The flag file can be in the web root of the internal container, since it can be reached only after a successful exploit of the vulnerability. The flag is owned by root, with 444 as permissions.

Another factor that has an impact on the generation of the environment is the attack vector. While an exploit requiring a *network* attack vector is directly executable by interacting with the container with standard networking tools such as `curl`, if the attack vector is declared as *local* the vulnerability requires the attacker to already have a form of access to the target system: to reproduce this precondition, the generated environment exposes an SSH endpoint for a low-privileged default user, called *ctfuser*, with known credentials. In this way, the attacker (or, in this case, the *evaluator*) is able to access the system and obtain an interactive shell in it, reproducing the real-world scenario required by the vulnerability.

Since the classification is produced by an LLM constrained only by a structured output schema, it is possible for the model to return a result that is syntactically

valid but internally inconsistent. For example, specifying a flag placement that assumes a privilege level the exploit never actually reaches. To mitigate this issue, the classification report is passed through a deterministic consistency check, before being accepted by the pipeline.
These checks are performed along three different dimensions:

- **Flag path shape**. The format of `flag_path` must match the declared vulnerability class. SQL injection classifications must produce a database reference (Database: X, Table: Y, Column: Z). SSRF classifications must produce an internal URL and must set `requires_secondary_container=true` with a non-empty `secondary_container_spec`. Every other vulnerability class must produce an absolute filesystem path.

- **Flag readability and execution context**. If the flag's permissions grant read access to its owner only, and the declared `flag_owner` differs from the `execution_context_user` (that is the identity under which the attacker first obtains code or command execution) then the classification is only consistent if either the attacker's identity is already root, or `kill_chain_steps` accounts for an explicit privilege-escalation step (i.e., `kill_chain_steps` ≥ 2). Without this check, the model could for example classify a command-injection vulnerability as a single-step exploit (`kill_chain_steps` $= 1$) while placing the flag under root ownership with owner-only permissions: such configuration is theoretically unreachable given the declared attack path.

- **Out-of-band interaction and kill chain length**. If the exploit is declared to require out-of-band interaction (`requires_oob_interaction=true`, such as a blind callback or DNS exfiltration), the kill chain must include at least one additional step beyond direct exploitation, since triggering and then retrieving the callback are logically distinct actions.

When one or more of these checks fail, the pipeline does not discard the classification outright. Instead, the exact list of detected inconsistencies is fed back to the model as a corrective instruction, asking it to re-emit the full classification while fixing only the flagged issues and leaving every other field unchanged. This process is repeated for up to two iterations. If the classification still fails validation after these retries, the pipeline does not block: it logs the remaining inconsistencies to the run's final report and proceeds with the last produced classification.

3.2.3 Environment Code Generation

The next steps in the pipeline are related to the actual generation of the vulnerable environment. The agent receives the list of the services required to build the environment and the classification report: it generates the vulnerable environment

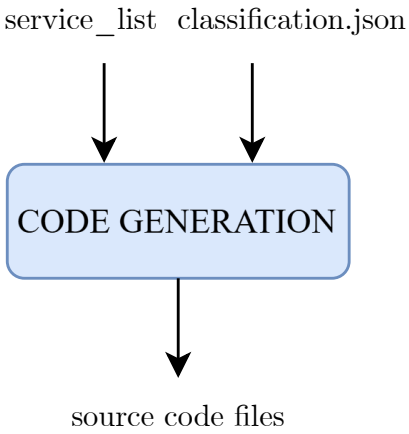


Figure 3.5: Input and output of the code generation phase

code, centered around a `docker-compose.yml` file that orchestrates the required services. The agent must respect some critical constraints in this phase, which will be validated in the next node (described in Section 3.2.4):

Basic Constraints

The agent must introduce in the container all the services listed in the received input. It must also produce an environment that does not require a manual setup. Moreover, the agent is also prohibited to create a mock version of the service (i.e. a simple application that emulates the behavior of the target one, but that does not use the original software).

Misconfigurations

If the CVE requires specific misconfigurations in the application, the agent must deliberately introduce them. For example, if the vulnerability requires weak permissions in a configuration file (such as the possibility for any user to upload any kind of content in a file), the agent must ensure that it is possible to do so;

Vulnerable Feature Exposure

This constraint is needed to ensure the vulnerability is actually exploitable. If the vulnerable service is correctly installed and it is running in the system, it must be possible to interact with the service in order to exploit the vulnerability.

Network Isolation

When the CVE needs an internal service (such as an internal web server for SSRF or a database for an SQLi), the agent must setup an internal network that is isolated from the one directly reachable from the host machine. This prevents the flag to be directly readable without an actual exploit.

Realism

The agent is instructed to build the vulnerable environment in a way that resembles an actual running application. This implies that the resulting container is not an empty box containing a flag, but it is instead an application to which is possible to interact with. Notice that this is the only constraint that is never tested: this limitation is described more in detail in Section 6.2. The way in which this constraint is applied directly depends on the attack vector. Here are reported some examples:

- **Network-Based Exploit:** The agent must choose a realistic theme for the web server, create some working endpoints that return a realistic response and blend the vulnerable endpoint in a normal-looking functionality, For example, the agent may configure the application as an e-commerce web site, with a list of products and the possibility to sort or filter them;
- **Local Exploit:** The container must be populated with the content of an ordinary system, such as some users unrelated with the vulnerability, a realistic shell history file and some configuration files. The permissions of unrelated files should follow the standard best practices for that kind of system. Furthermore, as introduced before, the environment must contain SSH and permit the login with the low-privileged user `ctfuser`, in order to provide a way to emulate a local interaction.
- **Non-Exposed Service Exploit:** The agent should find a theme for the internal container that is coherent with the overall theme chosen, such as a configuration server. The internal service should expose some benign endpoints together with the one leading to the flag. Most importantly, the internal server must not expose any port to the host machine, in order to ensure its isolation.

Finally, the flag is placed inside the environment according to the specifications defined in the classification step. The flag placement is constrained too:

- The flag is created as the last step of the generation phase, in order to avoid being overwritten;

- The flag must not be placed inside a directory that is declared as a Docker volume, since in such case that directory would be re-initialized after any `docker compose up` command;
- The flag should never be directly reachable on a public, static endpoint. Since this is a critical aspect, a more specific check will be performed in a later stage of the pipeline.

3.2.4 Iterative Testing and Revision: Creating a Working Environment

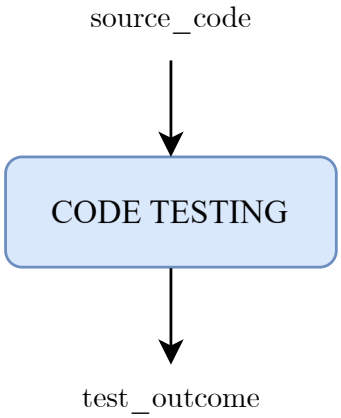


Figure 3.6: Input and output of the code testing phase

The code generated is then tested across two different dimensions, in order to check that the environment is correctly working.

Docker Tests

A dimension of tests is related to the actual Docker environment functionality. First, the Docker container is started by executing `docker compose up`: if the image generation produces an error, the agent reviews the logs and retries. Instead, if the image is successfully built, the agent waits to see if the environment runs without errors. Each container inside the environment is checked to see if its status is equal to `running`: if not, the code is generated again, otherwise the agent inspects the logs to evaluate if the environment is running without errors. After that, the services and versions chosen by the agent are reviewed against the list extracted during the web search. This step involves two distinct tests:

- the list of all the services required (both hard and soft types) is checked. If a service in that list is not found in the environment, the code must be generated

again.

- for each hard service in the environment, another check is used to verify that a vulnerable version was chosen. Also in this case, failing such test would require to generate again the code.

In both cases, the agent is prompted to review the code and fix it, with a prompt built based on the point of failure.
Finally, the flag placement is verified in a white-box fashion using the command `docker exec -u root cid sh -c test -f "flag_path"`, where `cid` is the container id and `flag_path` is the path at which the agent has chosen to place the flag in the classification phase. This command is used to verify that the flag is placed at its intended location. This last check is performed only for filesystem flags, since a similar approach cannot be used for database flags.

Network Tests

If the previous checks are passed successfully, other tests are performed: First, the agent verifies that all the containers in the environment that have exposed ports are actually reachable on the container network. Then, if there are internal containers, the agent checks that they are listening on an internal network and are not isolated. If all the previous checks are passed, it means that the Docker environment is running, the vulnerable service is correctly installed, and all the containers are reachable via the custom network defined in the compose file. What can still happen is that the vulnerable service is not configured or is not reachable. So, I implemented a final test to check that the exploit surface can be reached. It is important to notice that in this phase no exploit is actually attempted: instead, the agent first is prompted to generate possible probes to be launched against the proposed environment, given the source code. Then, the probes are actually executed and the agent is prompted to evaluate their output. If network error codes such as 404 or 500 are received, it may indicate a wrong configuration of the environment. In such cases, the agent reviews the interaction logs and evaluates if there is a misconfiguration. For example, if the error message is "Bad Gateway", the error code 500 indicates a wrong configuration, while a stack trace of the target component means that the service is alive, so the test is passed.

Revising the Code: Fixing the Environment

Any time one of the previous check failed, the pipeline enters the *CodeRevision* node. Here the agent receives the present version of the environment's code and the information about which test failed, together with the logs generated during the tests. The task assigned to the agent is to review the code and correct the

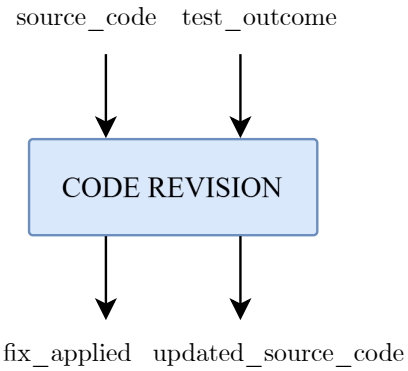


Figure 3.7: Input and output of the code revision phase

parts that lead to the test failure. For example, if the service inserted was wrong, the agent must fix the corresponding `Dockerfile` choosing the required version of the vulnerable service; while, if the test failed because of an isolated container, the agent must rewrite the `docker-compose.yml` in order to map the container on a custom network. The agent produces then an updated version of the code, and at the same time identifies the files that are no longer required and removes them. The updated code is produced in two sequential steps:

1. The agent is prompted to review the wrong code, together with the information about the failure, and must elaborate an explanation that justifies the failure and some possible strategies to fix the code.

2. Then, the output of this preliminary analysis is received again by the agent, that produces a structured output containing the updated code.

Finally, the Docker environment is re-launched and the tests start again.

I also implemented a mechanism to prevent the agent from getting stuck in a loop on errors of the same type: if the last three revisions only change the base image tag without solving the problem, an explicit hint is concatenated to the agent's prompt to force it to change its approach. The hint is a predefined prompt, in which the only dynamic part is represented by the erroneous image tag the agent is using.

If the tests fail for the tenth time, the pipeline is stopped to avoid being blocked on the generation of a single environment. I motivate this choice in Section 5.6.1. It is possible for the pipeline to enter again the revision loop in two circumstances: after the sanity check and after the debugging phase. It is important to notice that there is a difference in how the revision loop can be entered:

- Entering after the sanity check will not reset the iteration counter;
- Entering after the debugging phase will instead reset such counter.

The reasons behind this implementation choice will be better explained in the related sections of this chapter.

3.3 Sanity Check: Verifying if the Flag is Exposed

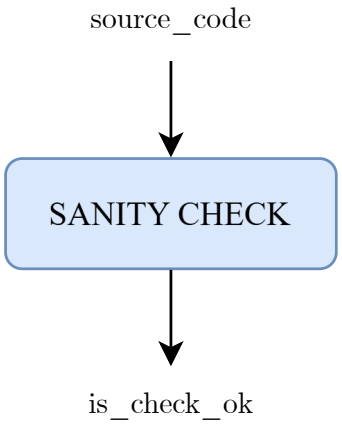


Figure 3.8: Input and output of the sanity check phase

The next phase is a crucial check to verify the soundness of our validation phase. Given the architecture of the system, some probes are deterministically defined and then executed to check if it is possible to retrieve the flag in a trivial way. Before executing the probes, the Docker environment is restarted, in order to have a clean context with respect to the generation phase. I introduced this node in the pipeline because, in the first experiments, it often happened that in environments containing a web server the flag was placed in its web root: consequently, it was possible to retrieve it by just using `curl http://localhost:port`, where port was the exposed port of the container. Other issues were present when the flag should be placed in an internal container, such as in an SSRF scenario: in such cases, the agent often generated environments in which it was possible to reach the internal container by directly interacting with an exposed API, without triggering any vulnerability. The sanity check mechanism prevents these situations by attempting to interact with all the exposed endpoints and by performing other trivial tests, such as basic curl operations. If the sanity check fails, it means that the flag was accessible in an unintended way: as introduced before, this event will result in a new revision of the code. Since an exposed flag requires only to fix a minor part of the environment (that has indeed passed all the other tests), I chose to consider this as another revision of the same loop. Finally, if this check fails for the third time, meaning the agent is not able to place the flag in the correct position, the

pipeline is stopped.

3.4 Vulnerability Validation: Is the Environment Exploitable?

This section explains how the validation phase is developed in the agentic workflow. It is divided into two phases, a static part and a dynamic part, that directly involves the second agent, the *evaluator*.

3.4.1 Static Validation Baseline

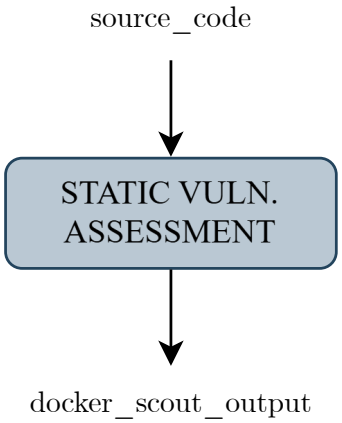


Figure 3.9: Input and output of the static validation phase

The environment developed during the previous step is now ready to be analyzed. First I perform a static evaluation through a specific tool, Docker Scout: this tool analyses the images associated to the environment and compares the installed software with a list of known vulnerable software and versions. It produces then a set of JSON files, one for each image, that contain the output of the tool, that is a list of all the CVEs that affect that image. Then, the CVE that I used as input for the pipeline is searched in each file. If there is a match, Docker Scout has flagged the environment as vulnerable to the input CVE. Otherwise, for the tool the environment is not vulnerable to that specific CVE. It is possible to see in Chapter 5 that there were different examples of false negatives (the environment was actually exploitable) and false positives. These last cases may be due to the static analysis performed by the tool: if the vulnerable version of the software is installed but for some reasons it is not reachable, neither the *evaluator* nor the PoC would be able to capture the flag. Such situation can still happen since some of the tests performed in the previous step were performed in the LLM-as-a-judge

setting, meaning that an LLM evaluates the work of another LLM: this can lead in some cases to hallucinations.

3.4.2 Dynamic Validation: Red Teaming

After this first validation, the environment is then launched again to test its vulnerability in a dynamic way.

Capture Oracle Soundness

As described in Section 3.2.3, during the generation phase a flag is placed in the environment as a placeholder string of the form `FLAG{verified_CVE-X-Y}`, where `CVE-X-Y` represents the unique identifier for the vulnerability.

The problem This value is deterministic and derivable from the environment, and the *evaluator* has white-box access to the source code files to do a reconnaissance phase before trying any exploit, as will be shown in the next paragraph. Consequently, the use of such placeholder as flag does not represent a strong evidence of a successful exploitation, since the agent may reproduce that value without exploiting the vulnerability.

The solution: a runtime nonce To enforce the validity of the evaluation mechanism, before executing the *evaluator* agent, the placeholder flag is replaced with a runtime random *nonce* (i.e. a random string, in the form `FLAG{`*nonce*`}`), which is different for each execution. This new value is directly injected in the running container using `docker exec` and it is never stored in files that can be read by the agent. Using this strategy, the only way the agent may be able to return the flag is through a real exploit.

The support level for this mechanism changes depending on the flag position in the environment:

- **Filesystem flags.** The fully supported case: the content of the flag file is overwritten by redirection, in order to preserve the original inode. Then, the file owner and permissions are re-applied according to the instructions determined during the classification phase, therefore the accessibility level of the flag will be consistent.
- **SSRF scenario flags.** A more complex mechanism, since the flag path is not a priori known: some possible paths through the flag file are determined starting from the URL and from common web directories (such as `/var/www/html`, `/usr/share/nginx/html`), until the correct path is found. After that the file content is overwritten with the nonce, an HTTP request is executed towards

the internal container. In this way, it is possible to check that the returned value corresponds with the one just injected: this step is needed because in some cases the application may return a cached value. If this check fails, the system falls-back to the deterministic placeholder, as a non-verified injection would represent a compromised oracle.

- **Database flags.** The nonce-injection mechanism is not supported for database flags: in such cases, the default placeholder is kept as flag. This choice represents a known limitation in the methodology that will be discussed more in detail in Chapter 6.

In any case, the outcome of the injection phase is registered in a boolean flag: in this way, it is possible to distinguish among environments verified using the nonce and environments verified with the placeholder.

Agent-Based Validation: the Evaluator

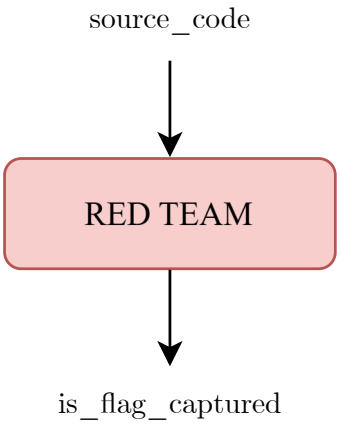


Figure 3.10: Input and output of the red team phase

After the nonce is injected, the *evaluator* is executed in the environment. The task of this agent is to capture the flag by exploiting the vulnerability emulating an interaction by network. It is important to underline that the objective of this phase is to demonstrate that the environment is vulnerable, and not to develop the best possible red team agent. Given this precondition, it is possible to understand why the *evaluator* was developed as a white-box agent. This means that the agent has full visibility over the source code of the environment, along with all the configuration files. Moreover, an automatic reconnaissance of the environment is performed before starting the agent: this procedure finds the ports the environment is exposing to the host, the possible internal containers and the Docker networks. This information, together with some details produced in the classification phase

(such as the vulnerability class, the exploit mechanics, the context of execution etc.) is assembled in a report that is received by the agent in its prompt. In this way, the agent will not waste time and tokens in retrieving information that is already available. Furthermore, at the same time the agent's working directory is also created, containing some auxiliary scripts that may be useful to interact with the target environment:

- a script to send HTTP requests;
- another script to start a simple HTTP server for out-of-band interactions;
- one last script, analogous to the second one, to receive raw TCP connections.

The *evaluator* is executed inside a sandboxed environment, in order to limit its capabilities:

- the agent cannot directly interact with the Docker daemon (such as by using `docker exec` or `docker compose`);
- the access to the configuration files and to the source code of the environment is in read-only mode;
- the agent can write only files inside its working directory;
- the file containing the injected flag nonce cannot be accessed by the agent.

The prompt provided to the agent is dynamically built based on the vulnerability class, determined during the classification phase. For each vulnerability family (such as remote command execution, path traversal, information disclosure, etc.) some possible techniques to attempt the exploit are defined, in order to help the agent to try different approaches after a failure. As mentioned, in case of local exploits the agent is able to access an SSH endpoint to locally connect inside the environment as a low-privileged user.

After the vulnerability is identified, the agent must produce a python or shell script (`final_exploit.py` or `final_exploit.sh`) that is able to reproduce the exploit and autonomously print the flag. To prevent the agent from cheating, especially for the cases in which the nonce injection is not supported, the output script is scanned using regular expressions to check it contains neither Docker commands nor a hardcoded flag string. If the script is evaluated as valid, it is executed against the environment. Finally, the flag string is searched in the standard output and in the standard error: if the flag is present, the agent has successfully exploited the vulnerability and the pipeline is stopped. Otherwise, the *forger* agent will determine the most likely reason that lead to the exploit failure, as explained in the next section.

3.5 Debugging After a Failed Exploit

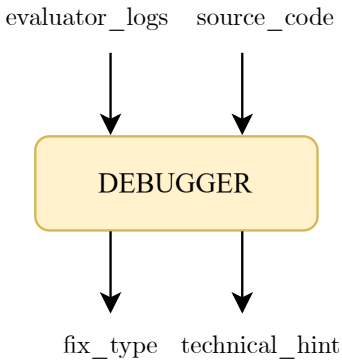


Figure 3.11: Input and output of the debugger phase

As mentioned in the previous section, this last node of the pipeline can be entered only after the *evaluator* failed an exploit attempt. The *forger* agent, in such cases, receives the execution log of the *evaluator* and the source code of the environment: based on that information, the agent attributes the failure to either the *evaluator* agent or the *forger* agent itself and proposes a possible solution. In particular, there can be two kinds of situations:

- The *evaluator* did not understand how to exploit the environment, either because it got the wrong exploit path, it reached the timeout or for inherent capability limitations;
- The exploit procedure was correct but the environment was misconfigured, so it was impossible to retrieve the flag.

In the former case, the solution is a *Soft Fix*: the *forger* develops a technical hint about the correct way to exploit the vulnerability, that will be injected at the end of the *evaluator*'s prompt. Then, the pipeline enters again the *RedTeam* node, that will be executed again with the updated agent's prompt. Instead, in the latter case the proposed solution is a *Hard Fix*: the agent suggests a modification that must be applied to the environment's source code, in order to let the vulnerability be exploitable. Then, the pipeline comes back to the revision loop: since the previous version of the environment was considered wrong, I reset the iteration counter, in order to give more possibilities to the *forger* agent to build a fixed version of the environment.
The output of the *Debugger* node is constrained to match the schema reported in the Appendix (Section A.6).

This debugging loop can be performed up to three times: if the exploit fails again after the third attempt, the pipeline is stopped and the workflow ends.

The effectiveness of this mechanism is documented more in detail in Section 5.6.2.

3.6 PoC-Based Validation

The last validation metric (Table 3.1) adopted to validate the environments generated with the pipeline is represented by the adaptation of known Proof of Concept (PoCs) scripts (specific for a given CVE) in order to let them retrieve the flag inside the environments produced by the pipeline. This metric was computed after the environments were generated, in a different time with respect to the main agentic workflow.

3.6.1 Retrieving the PoCs: The Scraper

First, I developed a simple script that searches the web for available PoCs. This script receives as input a list of CVE-IDs and looks for a PoC in the following repositories:

- **Vulhub:** An open source platform containing pre-configured Docker environments, that was already mentioned in Section 2.5.1. It often contains a PoC related to vulnerable environments[25].

- **Exploit-DB:** A public archive of exploits and PoCs that is maintained by the online cybersecurity community. It is used as a reference for analysis and checks on known vulnerabilities[34].

- **Trickest:** A platform designed to automate offensive security procedures, that includes updated collections of CVEs and public PoCs[35].

- **Github:** One of the most known platforms to host software repositories, that contains many projects related to cybersecurity, including exploits, PoCs, and community-developed tools to demonstrate and validate vulnerabilities[36].

The script always checks the first three sources mentioned above, while it falls back on searching in Github only if no PoC was found there. A *PoC-Dataset* is then generated: it contains a JSON file for each of the vulnerabilities present in the input list. Each file contains a field to indicate the specific CVE-ID and a list of the PoCs retrieved, reporting for each PoC information such as the original file name and the repository from which it was fetched, together with the actual PoC code.

3.6.2 PoC Adaptation and Execution

When the PoC dataset is generated, it is possible to run another script that autonomously adapts and executes each PoC against the corresponding environment. The adaptation phase is performed by relying on an LLM, that receives in the prompt the PoC code and the environment source code, along with some instructions to correctly generate the adapted script. This is not a trivial task, since the LLM must be able to adapt the endpoint names and ports in the original PoC to match the actual structure of the local environment. If the adaptation is successful, the environment is started: also in this context, the nonce injection procedure described in Section 3.4.2 is performed, in order to prevent a wrongly adapted PoC that prints the placeholder flag to pass the test. The outcome is checked in the same way as the final exploit produced by the *evaluator* agent: the adapted PoC is scanned, looking for hardcoded flag strings or Docker commands, and then it is executed. If the flag string is found in the standard output or in the standard error, the environment is classified as vulnerable.

Chapter 4

System Design and Implementation

While the previous chapter described the pipeline from a methodological point of view, this chapter describes the implementation details chosen to realize it.

4.1 System Architecture

The system is composed of two distinct agentic components, integrated into a single pipeline but implemented using different tools. The *forger* is implemented as an explicit state graph using *LangGraph*[31], a Python library that allows to define a control flow as a set of nodes (Python functions) and edges (transition conditions) on a shared state. This choice, that will be better motivated in Section 4.2, reflects the need for a strict order of operations and security mechanisms against infinite loops. The *evaluator*, on the other hand, is implemented using for the entire red teaming phase *Opencode*[32], a command-line agentic runtime that can autonomously execute bash commands, read and write files, and iterate on a natural language goal. The *evaluator* is not modeled as an internal LangGraph node: it is instead an external process invoked by the *RedTeam* node, that waits for the *evaluator* to complete its execution. This implementation choice reflects the different nature of the two tasks: the *forger* requires control and a typed state that can be verified, while the *evaluator* requires autonomy in exploration in an environment that is in any case isolated during the execution. Different models can be used for the *forger* and the *evaluator*, but the default configuration uses `DeepSeek-V4-Flash`[37] for both, as it is the most cost-effective option for large-scale experiments. Other models were tested during the experiments, but preliminary tests on a subset did not show substantial differences, so the cost of using more expensive models was not justified. In any case, the models were always

invoked through the same interface, that interacts with Openrouter[38]. The shared state between the *forger* nodes is defined as a Pydantic[33] model, as well as all the structured outputs required from the language models (classification, generated code, test results). This choice is discussed in more detail in Section 4.3.

4.2 Agent and Workflow Implementation

4.2.1 The Graph

Each phase of the pipeline described in Chapter 3, such as the classification or the code generation, corresponds to a node in the graph. In this graph, the transitions between nodes are deterministic: in some cases the transition is unconditional (such as the transition from *Classification* to *CodeGeneration*), while in other cases the transition is determined by the output of a deterministic routing function. I preferred this over an unconstrained agent, that would be able to decide each time which action to take, for two main reasons:

- generating a working Docker environment requires a strict order of operations (it would not make sense to test the code before it has been generated, nor to execute the red team before the environment is able to run correctly);
- an explicit graph allows to implement security mechanisms against infinite loops (such as the existing limits on test iterations and debugger retries) as deterministic properties of the graph itself, rather than relying on the discipline of a prompt to enforce them on an LLM.

As mentioned, each node in the graph is implemented as a Python function, that receives the current state (*OverallState*) and returns a dictionary of partial updates to the state. The routing functions created to handle conditional transitions do not introduce any other application logic: they only inspect the milestones contained in the current state and return a textual label that is used by LangGraph to determine the next node to execute. The separation between the generation of the milestones and the interpretation of them makes each node independent from the others, allowing to easily add new nodes or modify the existing ones. Moreover, this independence property of the nodes allows to implement a mechanism of partial execution, that can be used to resume the pipeline from a specific node in case of failure, without having to re-execute the entire graph from the beginning.

4.2.2 Partial Executions and Milestones Reset

As introduced in the previous section, the graph is designed to allow partial executions, that can be used to resume the pipeline from a specific node. To

exploit this feature, the system persists specific components of the state (such as the classification report, the generated code, and the milestones) at designated checkpoints throughout the pipeline, as it will be better explained in Section 4.9. This mechanism is implemented using a set of boolean flags, that are checked by the first routing function in the graph, which is evaluated right after the initial setup node. When a flag is set to true, the persisted state artifacts are loaded from disk before the graph is invoked, and the routing function directs the execution to the first node that has not yet been completed. A possible use-case for this feature is to allow the execution of the *evaluator* with a different prompt on a working environment that was generated in a previous run.

An important feature is represented by the way in which the milestones are handled during the generation of the environment, in particular regarding the ones that are related to the test performed over the environment in the *CodeTesting* phase (Section 3.2.4). After each test is passed, the corresponding milestone is set to true. If any of the next tests fails, the code must be generated again: consequently, the milestones related to the tests must be reset to false, to ensure that the next execution of the graph will re-run all the tests from scratch. This mechanism is implemented also after a failed sanity check or after a *hard fix* invoked by the *Debugger* node (Section 3.5).

4.2.3 Running the Evaluator

As previously mentioned, the *evaluator* is not implemented as a node in the graph: it is instead invoked as an external process by the *RedTeam* node, that waits for the *evaluator* to complete its execution. In particular, this node is responsible for preparing the execution environment for the *evaluator* by setting up the opencode configuration (Section 4.5), by injecting the runtime nonce (Section 4.6), by performing the automatic reconnaissance phase (Section 4.7), and by defining the prompt that will be used by the *evaluator* (Section 4.8). Then, the node invokes the *evaluator* using the `subprocess` module, and waits for its completion for a maximum of 600 seconds. If the *evaluator* does not complete its execution within this time frame, the node will terminate the process and mark the execution as failed. Therefore, it is important to notice that if the timeout is reached, it does not strictly mean that the environment is not vulnerable. It is a design choice to consider such event as a failure, since I cannot grant unlimited time and resources to the *evaluator*.

4.3 State Representation and Structured Outputs

The entire system relies on a shared state object defined as a Pydantic model, *OverallState*, which collects the information produced by each node throughout the pipeline execution. This includes metadata about the CVE (ID, web search results, classification), the generated environment (code, Dockerfile, docker-compose configuration), execution milestones (boolean flags indicating which validation steps have been completed), and cumulative statistics (such as iteration counts). By representing the state as a typed Pydantic model rather than as a free-form dictionary, the system is more robust to errors and easier to validate. The schema representing the *OverallState* is reported in the Appendix (Section A.2).

This design choice extends beyond the state object itself. Since the pipeline must apply deterministic checks (such as the ones described in Section 3.2.4), each expected output from the agent is also defined as a Pydantic schema. The output generated is constrained to match the corresponding schema, rather than interpreting the text produced freely. The constraints on the output are enforced in a textual way, by injecting the format instructions produced by a *PydanticOutputParser* into the system prompt. The parsing of the response is then performed using a custom parser: the model, even if explicitly instructed to produce only JSON, occasionally returned text preceded by preambles or enclosed in Markdown blocks. The parser implemented is therefore tolerant, scanning the text character by character for the first opening curly bracket and attempting to parse the JSON from that point.

4.4 Docker Orchestration

The interaction with Docker Compose is centralized in a utility module that exposes functions for starting, stopping and inspecting the generated environments. I chose to use Docker Compose for two main reasons: first, it allows to define the entire environment in a single declarative file, that can be easily generated and modified by the *forger*; second, it provides a higher-level abstraction that simplifies the management of multiple containers and networks, which is essential for the complex environments generated by the pipeline. Each environment is started using `docker compose up -d -build`, with an explicit timeout after which the process is terminated. In case of timeout, a cleanup is performed using `docker compose down` to avoid leaving the Docker daemon in an intermediate state, before reporting the failure. Since the completion of that command does not guarantee that the internal services are ready (many of them complete the initialization after the main process has started), a waiting phase is implemented: the system polls the exposed ports, that are dynamically extracted by inspecting the network settings of the

containers. A raw TCP socket is preferred over application-level checks (such as HTTP requests), because the pipeline must be protocol-independent. When no exposed ports can be found using this approach (which can happen for non-standard environments), the system falls back to a fixed-time wait. When any execution ends, the environment is stopped and the resources are explicitly released using `docker compose down -v -remove-orphans`.

4.5 Evaluator Isolation

Since the *evaluator* is a fully autonomous agent that can execute arbitrary bash commands, it is essential to introduce some constraints on its capability and on the environment in which it is executed, to ensure that it cannot bypass the validation mechanism and read the flag directly from the container. This is achieved through a combination of runtime permissions, process environment filtering, and post-execution checks for cheating attempts.

Runtime permissions. Before each execution of the *evaluator*, a configuration file `opencode.json` is generated. This file is used to configure the Opencode runtime, enabling basic tools (`bash`, `read`, `write`, `edit`, `glob`, and `grep`) and explicitly blocking through pattern matching the execution of specific bash commands, such as `docker exec *`, `docker compose *`, `rm -rf *` and `git *`. Also the access to external directories unrelated to the *evaluator*'s task is blocked. These constraints are used in order to prevent the agent from modifying files that are outside its scope (for example the source code of the pipeline or other system files) and also to prevent a possible interaction between the agent and the Docker daemon, which may let the agent discover the flag without trying the exploit. It is important to underline that the *evaluator* has complete visibility over the environment files, since it was implemented as a white-box red-team agent, as discussed in Section 3.4.2.

Process environment creation. The *evaluator* is executed in an environment containing a smaller set of variables with respect to the one of the parent process. A Python function builds the execution environment by filtering the parent process' environment using a combination of regular expressions and an explicit blacklist containing known variable names, such as secret keys (for example the Openrouter key used to invoke the language models) and variables related to the Docker daemon. This last constraint is established because if the agent is able to access the Docker variables, it may be able to interact with the remote daemon without needing to use `docker exec *`, that was blocked by the previous constraint.

Cheating detection. After the *evaluator* has produced a final exploit script to capture the flag, the script itself is scanned to ensure it contains no hardcoded Docker command, nor the hardcoded flag string. While this last cheating attempt would be ineffective thanks to the mechanism described in Section 4.6 in the majority of cases, as described in Section 3.4.2 the flags placed in a database would still be represented by the placeholder string: therefore, the agent may still be able to produce a script that only prints the placeholder and cheat, without such kind of checks.

4.6 Nonce Injection

The implementation of the mechanism that replaces the placeholder flag with a random nonce, that was introduced in Section 3.4.2, is centralized in a Python function. For filesystem flags, the nonce and the file path are provided as environment variables to the command `docker exec -u root` using the `-e` flag and they are referenced in the script using `$P` and `$V`, instead of being directly interpolated in the command string. This choice prevents quoting problems and potential injections in case of a path containing special characters. The flag overwriting is performed using redirection (`printf "%s" "$V" > "$P"`), in order to preserve the original inode and metadata. The owner and the permissions of the flag file are in any case applied again using `chmod` and `chown` to ensure the flag accessibility level corresponds to what was specified in the classification phase (Section 3.2.2). For SSRF scenarios and database flags, the logic is already described in Section 3.4.2.

4.7 Automatic Reconnaissance

The autonomous reconnaissance phase, whose logic was already described in Section 3.4.2, is implemented using a Python function. This function uses `docker inspect` to analyze the containers related to the target environment and distinguishes between the ones with port bindings (that are directly reachable) from the ones with no exposed ports (which may be potential SSRF targets). The result is summarized by another function in a JSON format and it is then injected in the prompt. The auxiliary scripts are written by a different function. Such scripts are provided to the *evaluator* in order to prevent the agent from committing syntax errors, that would lead to a waste of time and tokens, and to prevent the creation of processes in the foreground, that would block the entire session.

4.8 Prompt Design

In some stages of the pipeline, the *forger* agent receives a prompt that is dynamically generated depending on the classification produced for the specific CVE. The most important example of dynamic prompt received by the *forger* is the prompt used to generate the environment's code. In particular, this prompt is built starting from an initial section that is always present. Then, based on the vulnerability class, the corresponding flag placement rule (defined in Section 3.2.2) is appended to the prompt. After that, some instructions related to the attack vector are injected and, finally, further instructions on how to generate a realistic environment, determined based on both the attack vector and the vulnerability class. I adopted this implementation strategy because using a single, long, static prompt would contain many instructions that are inconsistent or irrelevant with respect to the ones that are truly needed by the *forger*. A dynamic prompt reduces this kind of risks. Other examples of dynamic prompts are the ones generated during the revision loop, that contain the error generated by the previous execution of the code together with the list of the fixes already attempted, and the prompt generated after a *hard fix* in the *Debugger* node: since the code and the misconfiguration it contains are not known in advance, that prompt is dynamic by definition.

Also the *evaluator* uses a dynamic prompt: in particular, a list of possible exploit instructions that are specific to the vulnerability category is defined, each one of them containing a sequence of concrete steps that can be attempted to exploit the vulnerability. For the scenarios in which the attack vector is defined as local, an alternative block that describes the SSH access as the entry point is always selected, regardless of the vulnerability class. This implementation detail reflects a methodological choice: the pipeline does not attempt to generalize a single behavior for any kind of vulnerability. Instead, it chooses the instructions that the agent should receive based on the vulnerability class that it is handling.

4.9 Logging and Results Persistence

When the pipeline is run on a CVE, any artifact produced during the execution is persisted in a dedicated directory. Inside it, the environment code (for example the compose file) is stored, together with a `logs/` subdirectory. This folder contains the actual execution trace of each node: the main files it contains are `web_search_results.json` and `classification.json` (produced in the information gathering stages), `logX.txt` files (generated after each test iteration performed in the revision loop), the `milestones.json` file and the `stats.json` file. These last two files have an important role that should be discussed more in depth.

The milestones file contains a list of boolean fields that are used to regulate the execution flow of the pipeline. In particular, it contains a field for each condition that an environment correctly generated should respect, such as `docker_builds` (true if the Docker image is built with no errors) and `code_hard_version` (true if the *hard services* are present in the correct version). There is a similar field for each test performed in the revision loop (that are described in Section 3.2.4): if any test fails, as mentioned in Section 4.2.2, all such fields are set to *false*. There are also other boolean fields related to other phases of the pipeline: `cve_id_ok` (true if the provided CVE-ID exists), `flag_protected` (true if the *SanityCheck* phase is passed), and `exploit_success` (true if the *evaluator* is able to capture the flag). The overall implementation of the milestones in the pipeline is reported in the Appendix (Section A.4).

On the other hand, the stats file contains some relevant metrics produced during the execution of the pipeline on a CVE. There are both cumulative metrics and boolean metrics. Among the former, there is a counter for the number of times the *Debugger* node was invoked and another one tracing how many test iterations were performed, while, among the latter, there is a field stating if the environment was considered vulnerable by Docker Scout and another field to indicate if the flag was captured by the *evaluator*.
The implementation of the schema that generates this file is reported in the Appendix (Section A.5).
Both these files contain very useful information to compute statistics about the performance of the pipeline, as it will be shown in Chapter 5.

4.10 PoC-Scraper Implementation

The scraping module (*Scraper*) sequentially looks for a PoC in the four sources described in Section 3.6.1. For Vulhub, it first retrieves the complete repository tree using the Github API (`/git/trees/master?recursive=1`) and extracts from there the folder related to the required CVE-ID. Then, from that folder, the *Scraper* downloads all the files whose name or extension is present in a list of common PoC names, such as `poc.py` or `exploit.sh`.
For ExploitDB, the *Scraper* downloads and parses the CSV index of the repository, that includes the `code` field, containing a list of associated CVE-IDs. If the required CVE-ID is found there, the corresponding file is directly downloaded.
For Trickest, the *Scraper* reads a Markdown file retrieved from the repository `trickest/cve-id`, that contains a list of URLs that link to Github repositories related to the CVE-ID. First, the URLs are extracted using regular expressions; then, the content of that repositories is inspected to retrieve files containing a PoC.

Finally, the Github search is used as a fallback. If the *Scraper* did not find any PoC, the Github search API is used to look for repositories containing the required CVE-ID in the name or in the description: if something is found, the relevant files are downloaded.
The PoC scripts retrieved in this way are then stored in a structured JSON file, that is specific for each CVE-ID. All the JSON files generated in this way are stored in a single folder, representing the `PoC Dataset` that will be used by the PoC-Runner module.

4.11 PoC-Runner Implementation

This last module is in charge of adapting the PoCs retrieved by the *Scraper* and run them on the corresponding local environment, to exploit the vulnerability and return the flag. In particular, it iterates over the `PoC Dataset` and, for each JSON file (that is associated to a unique CVE-ID), it executes the following procedure until a PoC in that file successfully captures the flag. First, an LLM is invoked to adapt a PoC from the JSON file: it is provided with the original PoC code and the content of the environment's compose file and other auxiliary files. The LLM's task is to produce a script that executes the same exploit mechanics of the PoC, but in which the target is `localhost:port`, where `port` is the exposed port in the target environment. Moreover, the LLM must also update the paths used to match the ones in the local environment (for example, if the exploit target in the PoC is the URL `/home/admin` and the environment contains `/usr/admin`, the code must be changed accordingly). The LLM should also change the PoC in order to remove required command-line parameters and hardcode them. Then, the script produced in this way is executed on the target environment and both the standard output and the standard error are saved in a file: if that file contains the flag, the adaptation was successful and the *PoC-Runner* can be executed on the next CVE in the dataset. Otherwise, if the adaptation fails or if the flag is not found in the output file, a maximum of two other attempts are performed, concatenating the execution output to the prompt used in the first attempt. From the output, the LLM should be able to recognize trivial errors, such as a wrong endpoint name, that were present in the first adaptation of the PoC.
It is important to notice that, before the adapted script is executed on the target environment, the nonce injection phase is performed also in this context, using the same procedure adopted before running the *evaluator*, that is described in Section 3.4.2. A particular behavior was observed before this mechanism was applied: many times, the LLM in charge of adapting the PoC simply produced a script that printed some information about how the exploit should work, and then directly printed the flag placeholder. This behavior happened more frequently in

this context with respect to what happened with the *evaluator*. For this reason, the introduction of a dynamic flag became crucial.

Chapter 5

Results

In this chapter, the results obtained using the methodology described in Chapter 3 are reported.

5.1 Dataset Overview

Overall, the dataset is composed of 226 different CVEs. Among them, in 150 different cases the *forger* managed to create a working environment, in which the *evaluator* tried to exploit the corresponding CVE to retrieve the flag. For 94 CVEs in this set of 150, a public PoC was retrieved using the *Scraper* module described in Section 3.6.1. In the following sections, different results will be computed across these different datasets. All the CVEs analyzed belong to 25 different CWE classes. The generation success rate, that is the number of environments the *forger* generated that reached the *evaluator*, is 66.4%. The success rate among the exploits attempted by the *evaluator*, on the other hand, is 38.0%.

Dataset Notation: in the following sections, the wording *Forger Dataset* (FD) refers to the dataset of 226 CVEs for which the environment generation was attempted, while the wording *Evaluator Dataset* (ED) refers to the subset of 150 CVEs for which an environment was successfully created. These datasets are represented in light blue and green in Figure 5.1, respectively. Finally, the yellow subset in the same figure will be referred to as the *PoC Dataset* (PD).

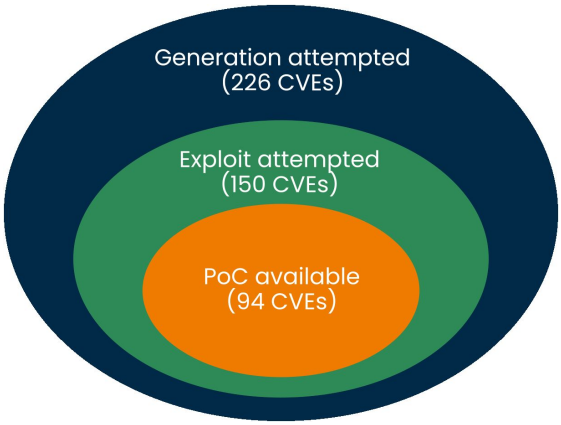


Figure 5.1: Representation of the CVE dataset

CWE ID	CWE Name	CVE(FD)	CVE(ED)
CWE-94	Code Injection	31	18
CWE-22	Path Traversal	28	18
CWE-78	OS Command Injection	20	14
CWE-89	SQL Injection	20	9
CWE-502	Deserialization of Untrusted Data	18	13
CWE-79	Cross-Site Scripting (XSS)	15	5
CWE-918	Server-Side Request Forgery (SSRF)	9	8
CWE-287	Improper Authentication	9	6
CWE-20	Improper Input Validation	7	4
CWE-200	Information Exposure	7	4
CWE-426	Untrusted Search Path	5	3
CWE-306	Missing Authentication for Critical Function	5	4
CWE-119	Buffer Overflow	5	4
CWE-434	Unrestricted Upload of File with Dangerous Type	5	4
CWE-284	Improper Access Control	4	4
CWE-400	Uncontrolled Resource Consumption	4	4
CWE-611	Improper Restriction of XML External Entity Reference (XXE)	4	4
CWE-190	Integer Overflow or Wraparound	4	3
CWE-285	Improper Authorization	4	3
CWE-352	Cross-Site Request Forgery	4	3
CWE-362	Race Condition	4	3
CWE-601	Open Redirect	4	3
CWE-74	Improper Neutralization of Special Elements Used by a Downstream Component	4	3
CWE-917	Expression Language Injection	3	3
CWE-269	Improper Privilege Management	3	3
Total		226	150

Table 5.1: Mapping between CWE-ID and CWE-Name, along with the number of CVEs for each CWE in the *Forger Dataset* (FD) and in the *Evaluator Dataset* (ED)

5.2 Comparison between Docker Scout, the Evaluator and the PoC

As introduced, the environments generated by the pipeline are validated using different approaches:

- Static analysis performed using Docker Scout;
- Public PoC adaptation performed by an LLM;
- Dynamic exploitation attempt performed by the evaluator.

In this section, the results reported will consider only the environments for which the PoC was available (i.e. the *PoC Dataset*). When the results achieved using Docker Scout and the *evaluator* are compared, the evaluator's outcome is treated as an approximation of a Proof of Concept. The *evaluator* was executed on these environment, despite the existence of a PoC, in order to measure its effectiveness compared to the PoC.

	PoC Successful	**PoC Failed**	**Total**
Scout Vulnerable	8 (8.5%)	23 (24.5%)	31
Scout Not Vulnerable	19 (20.2%)	44 (46.8%)	63
Total	27	67	94
	PoC Dataset		

Table 5.2: Comparison between static validation (Docker Scout) and the public PoC adapted.

	Evaluator Success	**Evaluator Failed**	**Total**
Scout Vulnerable	11 (11.7%)	20 (21.3%)	31
Scout Not Vulnerable	30 (31.9%)	33 (35.1%)	63
Total	41	53	**94**
	PoC Dataset		

Table 5.3: Comparison between static validation (Docker Scout) and dynamic exploitation outcome (*evaluator*) on the 94 generated environments for which a PoC was available

As it is possible to see from Table 5.2, many times the outcome of Docker Scout is unreliable: when the adapted PoC script captured the flag, only 8 times out of 27 (29.6%) Docker Scout flagged the environment as vulnerable. Similarly (Table 5.3), when the *evaluator* captured the flag, the static validation result was the same only 11 times out of 41 (26.8%). Most importantly, there were many false positives, that are cases in which Docker Scout returned an affirmative result, while neither the *evaluator* nor the PoC captured the flag. In particular, among 94 environments Docker Scout flagged an environment as vulnerable 31 times: among them, the PoC captured the flag only 8 times, while the *evaluator* managed to do so only 11 times.

	PoC Successful	**PoC Failed**	**Total**
Evaluator Successful	24 (25.5%)	17 (18.1%)	41
Evaluator Failed	3 (3.2%)	50 (53.2%)	53
Total	27	67	94
	PoC Dataset		

Table 5.4: Comparison between the *evaluator* and the public PoC adapted.

By contrast, Table 5.4 shows that the *evaluator* and the adapted PoC ended up in the same outcome 74 times out of 94. In particular, they both retrieved the flag in 25.5% of the cases, while only in 3 environments the PoC managed to reach the flag and the *evaluator* did not. In 17 environments (18.1%) the *evaluator* captured the flag while the PoC didn't: this may happen due to different reasons. The adaptation of the PoC may have been performed in a wrong way, since it is performed by an LLM, which may have not correctly understood the target environment. Another possibility is that the environment generated does not perfectly match the expected architecture for that specific CVE-ID: in such case, while the *evaluator* captured the flag because it was not limited by an existing script, the PoC could not be adapted to match a different exploit mechanism.

Static analysis alone misses real vulnerabilities, the *evaluator* is reliable

When the PoC captured the flag, Docker Scout called the environment vulnerable only in 29.6% of times (Table 5.2). Static detection both misses exploitable environments and raises false alarms, so it cannot stand on its own. This is why a dynamic check is needed. On the other hand, in more than three quarters of the experiments the *evaluator*'s outcome and the PoC's one were the same, suggesting that the dynamic evaluation is reliable.

5.3 Comparison between Docker Scout and the Evaluator

	Evaluator Success	**Evaluator Failed**	**Total**
Scout Vulnerable	22 (14.7%)	32 (21.3%)	54
Scout Not Vulnerable	35 (23.4%)	61 (40.6%)	96
Total	57	93	**150**
	Evaluator Dataset		

Table 5.5: Comparison between static validation (Docker Scout) and dynamic exploitation outcome (Evaluator) on all the 150 generated environments. In this case not always there was a PoC available, therefore I rely only on the agreement between these metrics

Extending the comparison between the results achieved with Docker Scout and the ones obtained by the *evaluator* over all the 150 environments in which the *evaluator* was executed (Table 5.5), it is possible to notice a pattern that is consistent with the results described in Section 5.2. Scout reaches an accuracy of 55.3% (83 out of 150) in predicting the exploitation outcome, a value lower than the accuracy that would reach a trivial classifier that outputs always "not vulnerable", which would correspond the 93 cases out of 150 in which the *evaluator* did not capture the flag (62.0%).

The relationship between Docker Scout's verdict and the outcome of the *evaluator* is not stable between the two sets: while in the subset of 94 CVEs the environments flagged by Scout as vulnerable have a lower success rate (35.5%) than the ones flagged as not vulnerable (47.6%), over the set of 150 this relationship is flipped (40.7% against 36.5%). This inconsistency reflects the fact that static validation alone, in this context, is not a reliable measure of the exploitability of the generated environments.

5.4 Forger Effectiveness based on the CWE

In this section, an analysis of the performance of the *forger* agent is reported, based on the CWE-ID associated to the input CVE. In particular, it is reported here a summary of the capability of the *forger* to generate a working environment with the characteristics described in Chapter 3 starting from the CVE-ID, regardless of the actual exploitation later attempted by the evaluator. Over the dataset of 226 CVEs, I consider a generation as successful when the pipeline reaches the red team

stage (the *evaluator*), but there is no clue if the generated environment is actually vulnerable: I only know the *forger* passed the tests described in Section 3.2.4, but it may have hallucinated during them. The exploit outcome is instead analyzed in Section 5.5. The dataset, as mentioned, covers 25 different CWE classes: the performance on the most represented ones are analyzed in detail, while the other classes are grouped depending on their *vulnerability family*. This concept is used to describe a group of CWEs with similar characteristics. It is introduced to compute statistics over the less represented classes. A map between the CWE-ID of the most represented classes and the associated name is reported in Table 5.1.

The Table 5.6 shows results that are not uniform among the different classes. The class more reliable to reproduce is CWE-918 (Server-Side Request Forgery), which presents only one container failure and no misconfigurations, with the highest rate of exploit phase reached (88.9%) and the smallest average number of test iteration required per environment (2.75), showing that such kind of environments are easier to build for the *forger*, despite the requirement of a secondary container. On the other hand, CWE-79 (Cross Site Scripting) is the most difficult class to represent: for more than half of the attempted CVEs (53.3%), the *forger* did not manage to produce a running environment, and only one third reaches the exploit phase. On the other classes, the *forger* has an intermediate performance (45-72% reached the exploit phase). CWE-94 (Code Injection) is the most represented class, but presents a high container failure rate (41.9%), resulting in the class with the highest number of environments that do not overcome that stage in the dataset, in absolute terms. CWE-89 (SQL Injection) presents a peculiar situation: while the 25.0% of the environments are rejected because of a container failure (a value similar to the ones of other classes), 30.0% is rejected instead because of a misconfiguration (the highest percentage in the dataset). This may reflect a difficulty experienced by the *forger* in linking the database service to the main container in a reachable state.

Vulnerability Families For the other CWEs, as introduced in the first part of this section, the aggregated rates for vulnerability family are reported, since the statistical support for individual CWE estimation was limited. The results are reported in Table 5.7.
The families considered and the CWEs they represent are the following:

- **Broken access & disclosure**: CWE-284 (Improper Access Control), CWE-285 (Improper Authorization), CWE-269 (Improper Privilege Management), CWE-306 (Missing Authentication for Critical Function);
- **Injection & parsing**: CWE-611 (Improper Restriction of XML External Entity Reference - XXE), CWE-917 (Expression Language Injection), CWE-74 (Improper Neutralization of Special Elements - Injection);

CWE-ID	n	ContFail (%)	Misconf. (%)	Exploit (%)	AvgIter.
CWE-918	9	11.1	0.0	88.9	2.75
CWE-502	18	22.2	5.6	72.2	5.20
CWE-78	20	25.0	5.0	70.0	7.44
CWE-287	9	22.2	11.1	66.7	5.89
CWE-22	28	25.0	10.7	64.3	5.35
CWE-94	31	41.9	0.0	58.1	6.29
CWE-89	20	25.0	30.0	45.0	7.33
CWE-79	15	53.3	13.3	33.3	7.43
Forger Dataset					

Table 5.6: Environment generation results for the most represented CWE classes (n≥8), considering all the 226 CVEs analyzed by the *forger*. `ContFail` refers to the environments for which was not produced a working environment (building or running failure); `Misconf` refers to the environments that run but were rejected because of a misconfiguration (either wrong service versions, wrong network setup, flag not present, services misconfigured); `Exploit` refers to the number of environments that reaches the red team stage (the evaluator); `AvgIter` refers to the average number of test iteration performed for that category of environments.

- **Request forgery & redirection**: CWE-352 (Cross-Site Request Forgery - CSRF), CWE-601 (URL Redirection to Untrusted Site - Open Redirect);

- **Resource, concurrency & input validation**: CWE-119 (Buffer Overflow), CWE-190 (Integer Overflow or Wraparound), CWE-400 (Uncontrolled Resource Consumption), CWE-362 (Race Condition), CWE-20 (Improper Input Validation);

- **Path & file handling**: CWE-434 (Unrestricted Upload of File with Dangerous Type), CWE-426 (Untrusted Search Path).

Vulnerability family	**n**	**Exploit (%)**
Injection & parsing	11	90.9
Broken access & disclosure	23	78.3
Request forgery & redirection	8	75.0
Resource, concurrency & input validation	24	75.0
Path & file handling	10	70.0
Forger Dataset		

Table 5.7: Environment generation results for the least represented CWE classes (n<8), grouped by the vulnerability family. The reported values do not consider the results already discussed in Table 5.6.

It is possible to notice that these vulnerability families show on average a higher success rate ($\geq$ 70%): this may be in some cases due to how the corresponding CVEs were selected, since these families include less common vulnerabilities with fewer known examples. Consequently, one possible explanation is that the chosen CVEs may be known representatives of such classes and be also known to the LLM for the same reason.

Build difficulty is uneven and CWE-driven

The forger is not equally good across vulnerability types: SSRF is reliable while XSS fails more than half the time. This means a single "generation success rate" hides large differences, and the weak classes are where the pipeline needs work.

5.5 Evaluator Effectiveness based on the CWE

In this section the focus is over the 150 environments in which the *evaluator* was executed, to analyze which classes are the most difficult to exploit when the environment is successfully created. The Table 5.8 shows the success rate for each vulnerability class. The worst result is associated to CWE-502 (Deserialization): while the environment is successfully generated in 72.2% of the experiments, only in 1 case out of 13 the vulnerability is successfully exploited. The *evaluator* most of the times is not able to build a working gadget chain, that is something consistent with the known difficulty in exploiting this kind of vulnerabilities. Path Traversal, OS Command Injection and SSRF scenarios are instead successfully exploited in half of the experiments, reflecting the fact that these vulnerabilities are in general easier to exploit. The class that presents the highest success rate is CWE-89 (SQL Injection): in the related experiments in fact the flag is retrieved in 6 environments out of 9 (66.7%). As mentioned in Section 3.4.2, for this kind of vulnerabilities the flag is not represented by a dynamic nonce, but it is instead the static placeholder `FLAG{verified_CVE-ID}`. However, while in general the *evaluator* may be able to retrieve the flag using its white-box access, in these cases a manual inspection of the final exploit script produced by the agent confirmed that the flag placeholder string was not hardcoded in the code, meaning that the flag was retrieved through actual execution of the exploit. This manual inspection was possible given the small number of environments that required it. On the other classes represented in the Table, the *evaluator* managed to capture the flag in at least one third of the experiments (success rate $\geq$ 33.3%).

CWE	**Attacked**	**Success rate (%)**
CWE-89 SQL Injection	9	66.7
CWE-22 Path Traversal	18	50.0
CWE-78 OS Command Injection	14	50.0
CWE-918 Server-Side Request Forgery	8	50.0
CWE-79 Cross-Site Scripting	5	40.0
CWE-94 Code Injection	18	38.9
CWE-287 Improper Authentication	6	33.3
CWE-502 Deserialization	13	7.7
Evaluator Dataset		

Table 5.8: Exploit success rate for the most represented classes, among the environments tested by the *evaluator* (n=150).

The least represented classes are again grouped by their vulnerability family in Table 5.9.

Vulnerability family	n	Success rate (%)
Path & file handling	7	71.4
Broken access & disclosure	18	33.3
Injection & parsing	10	30.0
Resource, concurrency & input val.	18	22.2
Request forgery & redirection	6	16.7
Evaluator Dataset		

Table 5.9: Exploit success rate for the least represented classes (n<8 in the main dataset), grouped by their vulnerability family, among the environments in which the *evaluator* was executed. These results are computed excluding the values already present in Table 5.8.

It is possible to notice that there is a difference with respect to the results achieved by the *forger*: the families for which was easier to generate an environment are not necessarily also simple to exploit. The family `Broken access & disclosure`, that was successfully generated by the *forger* in 78.3% of the attempts, is successfully exploited by the *evaluator* only in one third of the environments. A similar situation is represented by the `Injection & parsing` family. In particular, this family includes CWE-611 (XXE), that with 3 successful experiments out of 4 increases the value of the average result. The only family for which the *evaluator* is still performing well is `Path & file handling`, although it is composed of only 7 environments. Finally, the environments in the remaining two families are successfully exploited in less than one case out of four (22.2% and 16.7%), suggesting that resource exhaustion, race conditions and client-side forgery attacks are particularly hard to synthesize autonomously.

Easy to build is not the same as easy to exploit

The classes that are simple to generate are not always simple to attack: Deserialization builds well but is exploited only once out of thirteen, while SQL Injection is the easiest to exploit. Generation and exploitation have different bottlenecks.

5.6 Revision Loop and Debugger Loop Effectiveness

The pipeline includes two mechanisms that are able to condition its workflow: the *CodeRevision* node, described in Section 3.2.4, and the *Debugger* node, described in Section 3.5. The former operates over the full dataset population (226 CVEs), while the latter only on the environments that reached the *evaluator* stage (150 CVEs). It is recalled here that the revision loop can be executed a maximum of 10 times per CVE, while the debugging stage only 3 times. If after the debugging the *forger* determines a *Hard Fix*, the code is generated again and the iteration counter in the revision loop is reset to 0.

5.6.1 Impact of the Revision Loop

For the revision loop, considering only the last revision loop (excluding therefore the iteration performed before a potential *Hard Fix*), the average number of test iteration performed is 4.65 for a single environment. The main reason that lead to a test fail in the loop is a container building failure, with a total value of 965 container building failures, corresponding to an average of 4.27 failures per run, and 107 container running failures, that are on average 0.47 per run. Other failure causes are network or service misconfigurations (globally 305 cases, 1.35 per run on average).

Failure reason	**Tot**	**Environment avg**
Build failure	965	4.27
Run failure	107	0.47
Misconfiguration	305	1.35
Forger Dataset		

Table 5.10: Failure reasons in the revision loop, considering the full CVE dataset.

The iteration cost associated to the generation of an environment increases with the target complexity: the total number of iterations (considering also the ones executed before a *Hard Fix*) goes from an average of 5.88 for environments requiring a one-step kill chain to an average of 7.60 for environments requiring a three-step kill chain: this values are consistent with the hypothesis that multi-step environments are more difficult to reproduce.

Considering only the 76 CVEs that never reached the *evaluator* stage, that are the environments in which the failure depends only on the revision loop, there can

Kill chain steps	Avg iterations
1	5.88
2	5.70
3	7.60
Forger Dataset	

Table 5.11: Average iteration number for each environment (including iterations executed before a *Hard Fix*) with respect to the kill chain length.

be three possible situations:

- **Recurrent Error:** during the revision loop, the environment always failed the same test (for example, the image was never built because of a wrong version tag);
- **Different Errors:** each loop iteration failed for a different reason (for example, the first time because of a missing hard service, the second one for a wrong flag placement, etc.);
- **Mixed Behavior:** some errors occurred more than once during the loop, but the agent has attempted different strategies.

Error behavior	Environments	%
Recurrent error	66	86.8
Different errors	5	6.6
Mixed behavior	5	6.6
Total	76	100.0
FD \ ED		

Table 5.12: Error pattern in the environments in the *Forger Dataset* that are not in the *Evaluator Dataset*.

Most of the time, the maximum number of iterations is reached because of a *recurrent error*: in particular, in 66 cases (86.8%) the loop was stuck on the same error, while only in 5 environments (6.6%) the error is always different. Finally, the remaining 5 environments show a mixed behavior. From these results it is possible to notice that the bottleneck is the incapability of the agent in solving a single error source in the available iterations.

The 10-iteration limit is well chosen

The cap rarely stops runs that are still making progress: only 4.6% of the successful cases finish in the last two iterations. The real problem is the agent getting stuck on the same recurring error (86.8% of the failed builds), not a lack of attempts.

5.6.2 Impact of the Debugging Loop

Debugger invocations for each environment

Considering the 150 environments in which the *evaluator* was executed, 31 times (20.6%) the flag was captured at the first attempt, without executing the *Debugger* node. The remaining 119 environments (79.4%) invoke the *Debugger* at least once. In such cases, the *Soft Fix* is used more often than the *Hard Fix* (173 times and 57 times respectively). In 62 environments, only the typology *Soft Fix* was invoked, while in 10 environments only the *Hard Fix* one. In 47 environments, both the fix types are invoked: in fact, a single environment may require up to two fixes, that may be of a different kind.

Considering individually the environments, the effectiveness of the *Debugger* is similar between the two possible fix types: 22.0% of the environments that uses at least a *Soft Fix* captures then the flag, while the same happens in 21.1% of the environments in which at least a *Hard Fix* is used. These results are summarized in Table 5.13.

Category	**N**	**%**
No debugger invocation	31	20.6%
At least 1 invocation	119	79.4%
At least 1 Soft Fix	109	72.7%
At least 1 Hard Fix	57	38.0%
Soft Fix only	62	–
Hard Fix only	10	–
Both Fix types	47	–
Flag-capture rate, (at least 1 invocation):		
Environments with ≥1 Soft Fix	24 / 109	22.0%
Environments with ≥1 Hard Fix	12 / 57	21.1%
Evaluator Dataset		

Table 5.13: Frequency of debug-loop usage and final success rate, at the environment level (N = 150). The metric `Soft Fix only` refers to the environments in which only fixes of the kind *Soft* were invoked (similarly for `Hard Fix only`). The metric `Both Fix types` refers to the environments in which the *Debugger* was invoked two times (which is the maximum number of available invocations, as described in Section 3.5): one time the chosen fix was of the type *Hard*, the other instead the fix was *Soft*

Outcome of each Debugger invocation

Focusing on the individual *Debugger* executions (173 *Soft Fix*, 57 *Hard Fix* observed), it is possible to see the outcome of both the strategies. In general, after any type of fix, the possible situations can happen:

- **Soft Fix:** the fix was not effective (i.e. the *evaluator* did not capture the flag even after a fix) and the agent chooses to apply a *Soft Fix*;
- **Hard Fix:** similar to the previous one, but the agent chooses a *Hard Fix*;
- **Retry Cap exhausted:** two fixes have been already applied in the environment and the *evaluator* has not captured the flag yet. The pipeline is therefore stopped;
- **Flag Captured:** the fix was effective and the flag is captured;
- **Dead End:** after a *Hard Fix*, the *forger* did not manage to create another working environment and reached the maximum number of iterations in the revision loop. No exploits are attempted.

	Soft Fix		**Hard Fix**	
Outcome	**N**	**%**	**N**	**%**
Flag captured	19	11.0%	8	14.0%
Leads to another Soft Fix	64	37.0%	7	12.3%
Leads to a Hard Fix	30	17.3%	–	–
Retry cap exhausted	60	34.7%	28	49.1%
Dead end (rebuild never succeeded)	–	–	14	24.6%
Total invocations	**173**		**57**	
Evaluator Dataset				

Table 5.14: Immediate outcome of each debugger invocation, by fix type. For each possible outcome, the number of occurrences is reported for both *Soft* and *Hard* fixes

A *Soft Fix* leads to the capture of the flag in 11.0% of the experiments, while it leads to a *Soft Fix* or to a *Hard Fix* in 37.0% and 17.3% of the cases, respectively. The *Hard Fix* has an immediate success rate of 14.0%, but, on the other hand, in 13 cases out of 57 (22.8%) the generated environment after this fix is rejected because it does not pass the tests in the *CodeTesting* node (Section 3.2.4) in the available number of attempts. These results are summarized in Table 5.14.

Debugging recovers a useful minority of cases

Most environments (79.4%) need the *Debugger* at least once, and it leads to a captured flag in about one fifth of them. Soft and Hard fixes work about equally often, so the loop is a real second chance rather than a formality.

5.7 Other Metrics

In this section, the results achieved considering other relevant metrics are reported. Each subsection will consider both the complete dataset of 226 CVEs and the subset containing the 150 CVEs on which the *evaluator* was executed.

Win Rate: In the following tables, when it is reported `Win rate (E)` the meaning is the fraction of environments in which the *evaluator* captured the flag, among the ones in which it was executed. The wording `Win rate (F)` is used instead to denote the fraction of environments generated by the *forger* that reached the *evaluator*, among all the environments attempted.

5.7.1 Success Rate per CVE Release Period

Considering as metric the CVE release period and grouping them in three intervals (CVE released before 2019, after 2024, and between these years), it is possible to see different patterns in the performances of the *forger* and of the *evaluator*.
In Table 5.15 the former is reported: the *forger* success rate is similar in all the intervals, but it reaches its maximum for CVEs released until 2018 (72.1%) and its minimum for CVEs released between 2019 and 2024 (62.7%). The rate for the most recent CVEs (2025, 2026) is in between (68.8%). These results suggest that the oldest vulnerabilities may provide a more complete documentation and may also be present in the agent's knowledge base.

The *evaluator*'s performance, on the other hand, shows a different trend, as it is possible to notice from Table 5.16. The exploit success rate in fact increases with time. The most recent CVEs, even if released after the model's knowledge cut-off, are not more difficult to exploit: despite what could have been expected, the exploit success rate is not influenced by the model's memorization.

Period	Total	Evaluator	Win rate (F) (%)
Until 2018	68	49	72.1
2019–2024	126	79	62.7
2025–2026	32	22	68.8
Forger Dataset			

Table 5.15: Environment generation success rate per CVE release period.

Period	Total	Vulnerable	Win rate (E) (%)
Until 2018	49	17	34.7
2019–2024	79	31	39.2
2025–2026	22	9	40.9
Evaluator Dataset			

Table 5.16: Success rate per CVE release period.

Recency helps generation, not exploitation

Results suggest older CVEs are easier to reproduce, probably because they are better documented and more likely known to the model. Exploitation, instead, does not get harder for recent CVEs: even vulnerabilities released after the model's cut-off are attacked just as well, so memorization is not driving the exploit results.

5.7.2 Success Rate per Container Number

Also the success rate with respect to the number of containers in the environment shows different results depending on the agent that is analyzed. In Table 5.17, the results obtained from the generation phase are reported. The *forger* managed to generate environments composed of a single container in 75.7% of the attempts, while only in 57.0% when two containers were present. A similar value (60.0%) was obtained for environments composed of three containers, but there are only 15 samples in the dataset, so this value may not be reliable.

N. container	Total	Evaluator	Win rate (F) (%)
1	111	84	75.7
2	100	57	57.0
3	15	9	60.0
Forger Dataset			

Table 5.17: Environment generation success rate per container number in the environment.

From Table 5.18, it is possible to see that the exploit success rate is instead similar between the environments composed of one (38.1%) and two (40.4%) containers, but is much worse (22.2%) in the environments in which three containers are present.

Multi-container environments are a generation problem

A second container clearly lowers the build rate (75.7% to 57.0%) but barely touches the exploit rate. The cost of extra structure falls almost entirely on the forger, not the evaluator: linking and wiring services correctly, rather than attacking them, is the harder task.

N. container	Total	Vulnerable	Win rate (E) (%)
1	84	32	38.1
2	57	23	40.4
3	9	2	22.2
Evaluator Dataset			

Table 5.18: Success rate per container number in the environment.

5.7.3 Success Rate per Kill Chain Length

The kill chain length shows a similar pattern in the *forger*'s (Table 5.19) and in the *evaluator*'s (Table 5.20) performance. The generation success rate is comparable between environments requiring one (67.2%) or two (70.1%) steps, but becomes very low when the vulnerability requires three steps to be exploited. Considering the exploitation results, the success rate is quite stable between one-step (39.1%) and two-steps exploits (37.0%), while no environment requiring three steps is exploited by the *evaluator*, although only two environments of this kind were accepted by the *evaluator*. There are only two cases in which a four-step kill chain is required for the exploit, so they do not represent a relevant result.

Kill chain steps	Total	Evaluator	Win rate (F) (%)
1	137	92	67.2
2	77	54	70.1
3	10	2	20.0
4	2	2	100.0
Forger Dataset			

Table 5.19: Environment generation success rate per kill chain length.

Kill chain steps	Total	Vulnerable	Win rate (E) (%)
1	92	36	39.1
2	54	20	37.0
3	2	0	0.0
4	2	1	50.0
Evaluator Dataset			

Table 5.20: Success rate per kill chain length.

Three-step kill chains are a double bottleneck

One and two-step vulnerabilities behave similarly, but three-step ones are hard both to build and to exploit. Longer attack chains are the point where the whole pipeline struggles most.

5.7.4 Success Rate per Attack Vector (Network vs Local)

The majority of the dataset is composed of CVEs that require a *network* attack vector, with only few samples (5.7%) belonging to the class *local*. However, given the different approach that these types of vulnerabilities require, in Table 5.21 are reported the related outcomes.

Attack Vector	Network	Local
Total	213	13
Container Fail	56	2
Misconfiguration	17	1
Exploit Attempt	140	10
Win rate (F) (%)	65.7	76.9
Vulnerable	51	6
Win rate (E) (%)	36.4	60.0
Avg Iterations	6.01	4.45
Forger Dataset		

Table 5.21: Success rate per attack vector. In this table are reported both the performances of the *forger* and of the *evaluator*

Local CVEs look easier, but the sample is tiny

Local vulnerabilities score higher than network ones for both the forger and the evaluator. The dataset holds only 13 local cases, though, so this result is a hint to explore, not a firm conclusion.

5.7.5 Success Rate per Techstack

It is possible to consider as a metric also the main programming language used to generate each environment. The results discussed in this section consider only the most common programming languages that are present in the dataset. From Tables 5.22 and 5.23, it is possible to notice that Java is the most present class: despite being successfully generated in two third of the experiments, the related environments are exploited only in 26.2% of the cases, which is the lowest result achieved according to this metric. PHP presents similar results in the generation phase, but the exploitations in such environments are successful one time every three attempts on average. Both the *forger* and the *evaluator* offer better performances in C-based and Go-based environments, while environments written in Python are

successfully generated only in 56.5% of the attempts. At the same time, when the *evaluator* is executed on a Python-based environment, the flag is captured in 61.5% of the attempts.

Stack	Tot	ContFail	Misconf	Evaluator	Win rate (F) (%)
Java	63	19	2	42	66.7
PHP	54	15	4	35	64.8
C	25	6	0	19	76.0
Python	23	5	5	13	56.5
Go	10	3	0	7	70.0
Node.js	10	2	2	6	60.0
Forger Dataset					

Table 5.22: Environment generation success rate per underlying techstack.

Stack	Evaluator	Exploited	Win rate (E) (%)
Java	42	11	26.2
PHP	35	12	34.3
C	19	7	36.8
Python	13	8	61.5
Go	7	3	42.9
Node.js	6	2	33.3
Evaluator Dataset			

Table 5.23: Exploit success rate per underlying techstack.

Language results are tangled with the CWE

Java shows the lowest exploit rate and Python the highest, but the language is often tied to a specific vulnerability type (Java environments are frequently Deserialization, which is hard to exploit). The tech stack should not be read on its own, but together with the CWE.

5.7.6 Success Rate per EPSS Score (CVE Virality)

One last metric that can be investigated is the EPSS score (defined in Section 2.1.4). By analyzing the distribution of the environments in different score intervals, it is

possible to notice some interesting results, reported in Table 5.24. It is recalled here that the EPSS score measures how likely a vulnerability will be exploited in the wild, therefore it can be used also to measure how much a particular vulnerability is known. The EPSS score is represented by a decimal number between 0 and 1.

EPSS	Tot	Evaluator	Vulnerable	Win rate (F)	Win rate (E)
0.00–0.10	62	30	11	48.4	36.7
0.10–0.20	22	14	7	63.6	50.0
0.20–0.30	11	6	2	54.5	33.3
0.30–0.40	13	9	3	69.2	33.3
0.40–0.50	6	5	1	83.3	20.0
0.50–0.60	5	4	0	80.0	0.0
0.60–0.70	8	6	3	75.0	50.0
0.70–0.80	10	6	3	60.0	50.0
0.80–0.90	23	17	9	73.9	52.9
0.90–1.00	66	53	18	80.3	34.0
Sum	226	150	57		
Forger Dataset					

Table 5.24: Distribution of the generated environments according to their EPSS score. In this table are reported both the performances of the *forger* and of the *evaluator*

Considering the generation of the environments, the number of runs that reaches the *evaluator* increases with the EPSS score: starting from 48.4% achieved in the lowest EPSS score range, the *forger* managed to correctly generate 80.3% of the environments associated to the highest range. The trend is positive overall but non-monotonic, with some exceptions in the ranges containing less samples. The general pattern is consistent with the hypothesis that CVEs with a higher EPSS score are easier to reproduce, despite the inherent environment complexity.

Considering the *evaluator* performance, the exploit success rate does not show any recognizable pattern: the values go from 20.0% in range 0.40–0.50 to 52.9% in range 0.80–0.90, without a defined trend.

EPSS predicts building, not exploiting

A higher EPSS score goes with easier generation (from 48.4% up to 80.3%), likely because popular CVEs are better documented. It says nothing about exploitation: once the environment exists, the EPSS score does not predict

how hard the flag is to capture.

Key takeaway — What these results say about the thesis

Taken together, the results support the central claim only in a qualified way. Agentic AI can build a running environment in about two thirds of the cases (66.4%), but a running environment is not automatically a vulnerable one: the share confirmed as actually exploitable by the evaluator is 38.0%, and on the PoC subset the evaluator agrees with the public exploit in 78.7% of cases. So the system is able to automate *both* the construction and the dynamic validation of vulnerable playgrounds, but only the validated subset can be claimed as truly vulnerable. The static analysis alone (Docker Scout) is shown to be an unreliable substitute for this check.

The single most critical stage is the generation revision loop. Every failure mode that never reaches the evaluator concentrates there, and in 86.8% of those cases the agent is stuck repeating the same error rather than running out of attempts. The hardest cases for the whole pipeline are multi-container and multi-step (three-step) environments and, on the exploitation side, classes such as Deserialization. These are the natural targets for future work.

Chapter 6

Conclusions and Future Work

This thesis investigated whether agentic AI can automatically generate vulnerable playground environments starting only from a CVE-ID, and whether the resulting environments can be dynamically validated without relying on the application's source code or a public exploit. To this end, I designed and implemented a dual-agent framework: a *forger* agent reconstructs the affected service and its configuration, producing a containerized environment with an embedded flag, while an *evaluator* agent independently attempts to exploit the generated environment and retrieve the flag as an observable signal of exploitation.

6.1 Summary of Contributions

The proposed pipeline extends the previous baseline, which was limited to environment generation and static validation [3], in the following ways:

- it introduces a dynamic validation stage based on a CTF-style flag capture, employing an autonomous white-box red team agent;
- it operates under a weaker input assumption with respect to source-driven approaches such as CVE-GENIE [1], requiring only the CVE-ID;
- it extends the validation of the vulnerable environment requirements in the generation stages.

I evaluated the pipeline on 226 CVEs spanning 25 CWE classes, different release periods, tech stacks, and difficulty levels. The *forger* produced a working, testable environment in 150 of the 226 cases (66.4%), while the *evaluator* captured the

flag in 38.0% of the 150 environments it attacked. On the 94 environments for which a public PoC was available, the adapted PoC and the *evaluator* agreed on the outcome in 74 cases out of 94 (Table 5.4), which supports the reliability of the dynamic validation signal.

The comparison with static validation is the central empirical result of this work. Docker Scout reached an accuracy of only 55.3% in predicting the dynamic outcome over the 150 environments (Table 5.5), below the 62.0% that a trivial classifier always predicting "not vulnerable" would obtain. Its verdict was also not stable across the two evaluation subsets and it produced both false negatives (environments successfully exploited but not flagged) and false positives (environments flagged but never exploited by either the PoC or the *evaluator*). Taken together, these findings show that static analysis alone is not a reliable measure of exploitability for automatically generated environments, and that a dynamic, exploit-based oracle provides a stronger validation signal. It is possible to notice that the results provide evidence that agentic AI can be used not only to automate the construction of vulnerable playgrounds, but also to autonomously assess whether the generated vulnerabilities are actually exploitable.

6.2 Limitations

The results above should be interpreted in light of a number of limitations, which also motivate the directions discussed in Section 6.3.

Incomplete protection of the capture oracle. The soundness of the dynamic oracle relies on replacing the deterministic placeholder flag with a fresh runtime nonce that cannot be derived from the environment source code (Section 3.4.2). This mechanism is currently implemented for filesystem flags and, in a more elaborate form, for SSRF scenarios, but *not* for flags stored in a database (e.g., SQL injection targets), where the placeholder string `FLAG{verified_CVE-X-Y}` is retained. For these cases the oracle is therefore weaker: a white-box agent could in principle reproduce the placeholder without exploiting the vulnerability. The impact was mitigated in this work through a manual inspection of the final exploit scripts for the affected SQL-injection environments, which confirmed that the flag was retrieved through actual query execution and was not hardcoded. This manual check was feasible only because of the small number of database-flag environments, and does not scale. Complementary anti-cheating measures (the regular-expression scan for hardcoded flags and Docker commands) are similarly best-effort and can be circumvented, for instance through obfuscated or runtime-reconstructed strings.

Extending the nonce mechanism to database flags is technically more demanding than the filesystem and SSRF cases, and was deliberately left out of the scope

of this work. Unlike a file, which can be overwritten uniformly through `docker exec` regardless of the application built on top of it, a database record requires an engine-specific write (e.g., `mysql`/`psql`/`sqlite3` each have distinct client binaries, invocation syntax, and authentication mechanisms), together with the retrieval of valid credentials from the environment's configuration and a reliable parse of the `Database: X, Table: Y, Column: Z` reference produced by the classification stage. Given that only 9 of the 150 environments reached by the evaluator (6%) used a database flag (Table 5.8), the engineering cost of a general, per-engine nonce-injection mechanism was considered excessive with respect to its impact on the overall validation soundness, especially given the small size of this subset. This trade-off is revisited as future work in Section 6.3.

No guarantee that each phase is completed correctly. Because the pipeline is driven by probabilistic language models, the successful completion of a stage does not guarantee that its intended semantics were satisfied. In particular, some constraints are enforced only through prompt instructions rather than verified deterministically. Most notably, the *realism* of the generated environment, that is the requirement for the container to resemble an actual running application and not an empty box wrapped around a flag, is only requested in the prompts (Section 3.2.3 and Section 3.2.4) and never checked. Moreover, some validation steps are performed in an LLM-as-a-judge setting, in which one model evaluates the output of another, which can introduce hallucinations and accept environments that are misconfigured.

Flag capture does not certify the specific CVE mechanism. If the *evaluator* successfully captures a flag, the actual meaning is that there exists at least *one* exploitable path to reach the flag, but this does not necessarily mean that the exploited path corresponds to the mechanism of the target CVE. The *forger* may have introduced a different weakness in the environment or it may have placed the flag in a position that is reachable also through a different flaw with respect to the one described by the CVE. In such cases the *evaluator* would still retrieve the flag, so the environment would be labelled as vulnerable even if the intended vulnerability was not the one actually exploited. The current pipeline does not verify that the synthesized exploit triggers the specific root cause classified in Section 3.2.2, and this gap should be considered when interpreting the exploitation results as evidence of CVE-specific vulnerability.

The evaluator is used as a proxy for ground truth. The confusion matrices comparing Docker Scout with the dynamic outcome (Tables 5.3 and 5.5) treat the *evaluator*'s verdict as the reference outcome. The *evaluator* is itself a probabilistic oracle: a negative outcome may reflect a limitation of the agent, since building an

optimal red-team agent was not an objective of this work, while it is considered instead the consequence of a non-vulnerable environment. In particular, it is important to underline that the 600-second timeout, counted as a failure by design (Section 4.2.3), makes the reported exploitation rate a lower bound on the true exploitability. The strongest available reference, the public PoC, is only defined for 94 of the 150 CVEs in the *Evaluator Dataset* and is not uniformly distributed across CWEs, discovery periods, and difficulty levels.

Determinism, single-run measurements, and reproducibility. Each CVE was processed in a single pipeline execution. Given the stochastic nature of the underlying models, and the dependence of the information-gathering stage on the state of the web at run time, re-running the pipeline may lead to different outcomes for the same CVE. No variance analysis (such as multiple runs) is reported, and the web-search dependency introduces an input that is not fully reproducible and may, in some cases, expose the *forger* to existing environments or PoCs it could imitate. The main experiments also rely on a single model family (DeepSeek-V4-Flash); the claim that other models did not change the results substantially was not backed by a reported comparison.

Scope and dataset size. The pipeline targets vulnerabilities that are representable in a Linux-based Docker container, and a deterministic feasibility filter discards services outside this scope (Section 3.2.1); vulnerabilities requiring specific hardware, non-Linux platforms, or adjacent/physical attack vectors are therefore out of scope. Finally, although the overall dataset of 226 CVEs is sizeable, several breakdowns rest on small subsets (e.g., local attack vector with $n = 13$, three-container environments with $n = 15$, three and four-step kill chains), which limits the statistical power of the corresponding conclusions.

6.3 Future Work

The limitations above suggest several concrete directions for future work.

Generalizing the nonce oracle. The most immediate improvement is to extend the nonce injection mechanism to database flags, by writing the random string in the target record at runtime (as it currently happens for filesystem and SSRF flags), in order to have a uniform oracle mechanism for all the vulnerability classes. This would remove the need to perform a manual inspection for SQL-injection environments and make the capture oracle uniform across all flag types.

Certifying the CVE mechanism through differential testing. In order to address the risk that the flag is captured through an unintended path, the generated environment could be compared against a *patched* counterpart of the same software (a distinction already discussed in Section 2.1.6). If the synthesized exploit succeeds against the vulnerable image but fails against the patched one, this provides strong evidence that the exploit exercises the specific mechanism of the target CVE. The same differential setup could additionally serve as an automated, exploit-grounded ground truth, reducing the dependence on the *evaluator*'s own verdict.

Verifying realism and configuration. Rather than requesting realism only in the prompt, future versions could introduce an explicit verification step (for example an LLM-as-a-judge, or human evaluation on a representative sample) to check that the generated environment resembles a real application and that the intended misconfiguration is present. Replacing also LLM-as-a-judge checks with deterministic controls, when possible, would reduce the likelihood of errors propagating in the pipeline.

Stronger and different evaluators. Since a negative exploitation outcome may be caused by the *evaluator*'s limitations, future work could use multiple evaluators and report their overall outcome. In particular, introducing specific tools for the hardest classes to exploit, such as deserialization (CWE-502), would help distinguish between environment quality and attacker capability. Reporting results over multiple runs, with confidence intervals and metrics, would also quantify the overall pipeline capabilities.

Towards a self-updating benchmark. Finally, the dynamically validated environments produced by this pipeline can be used to build datasets with exploitability labels, to assess the reliability of static detection tools at scale, and to extend curated collections such as Vulhub [25]. Because the system only requires a CVE-ID, it is a natural candidate for a continuously updated benchmark of recent, post-cutoff vulnerabilities.

Appendix A

Some Relevant Code Definitions

In this section, the most important code definitions that were used in the pipeline are reported.

A.1 VulnerabilityClass

```
class VulnerabilityClass(str, Enum):
    rce = "rce"
    deserialization = "deserialization"
    path_traversal = "path_traversal"
    sqli = "sqli"
    ssrf = "ssrf"
    privesc = "privesc"
    auth_bypass = "auth_bypass"
    info_disclosure = "info_disclosure"
    xxe = "xxe"
    xss = "xss"
    csrf = "csrf"
    other = "other"
```

Listing A.1: Class implementation to define vulnerability classes

A.2 OverallState

```
class OverallState(BaseModel):
    model_name: str = Field(
```

```
        default="",
        description="Name of the model chosen for this agent run"
    )
    base_path: Path = Field(
        default=PROJECT_ROOT,
        description="The path of the root directory of the project"
    )
    forger_llm: ChatOpenAI = Field(
        default=None,
        description="LLM instance being used"
    )
    cve_id: str = Field(
        default="",
        description="The input CVE-ID"
    )
    web_search_result: WebSearch = Field(
        default=WebSearch(desc="", cwe="", attack_type="", services=[]),
        description="The result of the web search"
    )
    code: Code = Field(
        default=Code(files=[], directory_tree="", target_hostname=""),
        description="The generated file name, code and associated directory tree"
    )
    fail_explanation: str = Field(
        default="",
        description="Detailed explanation of why the Docker has failed testing"
    )
    revision_type: str = Field(
        default="",
        description="Type of revision to be applied to the Docker code"
    )
    revision_goal: str = Field(
        default="",
        description="Goal of the revision phase"
    )
    fixes: list[str] = Field(
        default=[],
        description="List of attempted fixes to the code"
    )
    platform_feasible: bool = Field(
        default=True,
        description="False if the service requires a platform that cannot be
    containerized in Linux"
    )
    platform_infeasibility_reason: str = Field(
```

```
47          default="",
48          description="Explanation on why the input CVE was considered
    impossible to represent on Docker and Linux"
49      )
50      stats: Stats = Field(
51          default=Stats(),
52          description="Various stats about the current workflow"
53      )
54      milestones: Milestones = Field(
55          default=Milestones(),
56          description="Milestones of the workflow, used to track its progress"
57      )
58      classification: CVEClassification = Field(
59          default=None,
60          description="Classification of the CVE: contains info like the CWE"
61      )
62      sanity_retry_count: int = Field(
63          default=0,
64          description="Number of times this step is repeated"
65      )
66      sanity_result: SanityCheckResult = Field(
67          default=None,
68          description="Result of the sanity check (e.g., is the flag exposed?)"
69      )
70      skip_to_red_team: bool = Field(
71          default=False,
72          description="Flag that forces to skip to the last step in the pipeline"
73      )
74      skip_to_scout: bool = Field(
75          default=False,
76          description="Flag that forces to skip to the docker scout check in the
    pipeline"
77      )
78      reuse_classification: bool = Field(
79          default=False,
80          description="Flag that forces to skip to the code generation phase"
81      )
82      reuse_web_search_and_code: bool = Field(
83          default=False,
84          description="Flag that forces to skip to the phase in which the code is
    tested"
85      )
86      debug_retries: int = Field(
87          default=0,
88          description="Number of times the Debugger node was executed"
```

```
    )
    debugger_action: str = Field(
        default="" ,
        description="The action chosen by the debugger (either 'Hard Fix' or 'Soft Fix')"
    )
    dynamic_hint: str = Field(
        default="" ,
        description="The hint the debugger chooses to give to the evaluator"
    )
```

Listing A.2: Class implementation of the *OverallState*

A.3 CVEClassification

```
class CVEClassification(BaseModel):
    cve_id: str = Field(description="The CVE-ID provided as input")
    cwe_id: str = Field(description="e.g. CWE-78")
    cwe_name: str = Field(description="The name commonly associated to a CWE-ID")
    vulnerability_type: VulnerabilityClass = Field(description="The type of vulnerability (according to the class defined) in which this CVE can be mapped")
    attack_vector: AttackVector = Field(description="The required attack vector (e.g. local, network)")
    requires_auth: bool = Field(description="Does this CVE require to be authenticated to be exploited?")
    vulnerable_component: str = Field(description="Brief description of the vulnerable component or endpoint")
    execution_context_user: str = Field(description="OS user under which the exploit runs, e.g. www-data, root, redis")
    exploitation_summary: str = Field(description="2-3 sentence plain-English description of how the exploit works mechanically")
    kill_chain_steps: int = Field(description="Number of distinct logical steps required to exploit the vulnerability and read the flag (e.g., 1 for direct RCE, 3 for Auth Bypass -> Token Extract -> RCE).")
    flag_path: str = Field(description="Absolute path, DB reference (Database: X, Table: Y, Column: Z), or internal URL")
    flag_owner: str = Field(description="The user in the system that should own the flag file")
    flag_permissions: str = Field(description="Octal string e.g. '400', or 'N/A' for DB flags")
    placement_rationale: str = Field(description="An explanation to justofy the flag placement decision")
```

```
16     requires_secondary_container: bool = Field(description="Does this CVE
       require more than one container?")
17     secondary_container_spec: str | None = Field(default=None,
       description="Description of the hidden container if needed")
18     tech_stack: str = Field(description="The primary language or framework
       of the vulnerable component (e.g., Java/Spring, PHP, Node.js, Python/Flask, C/
       Binary).")
19     requires_oob_interaction: bool = Field(description="True if the exploit
       typically requires setting up a reverse shell, DNS interaction, or a local HTTP
       server to catch callbacks.")
20     red_team_initial_hint: str = Field(description="A highly specific, 2-
       sentence actionable technical advice for the Red Team on how to approach this
       specific CVE.")
```

Listing A.3: Definition of the class used by the pipeline to generate the classification report

A.4 Milestones

```
1  class Milestones(BaseModel):
2      cve_id_ok: bool = Field(default=False, description="Does the provided
       CVE-ID exist in the MITRE CVE database?")
3      docker_builds: bool = Field(default=False, description="Do all Docker
       images get built correctly?")
4      docker_runs: bool = Field(default=False, description="Do all Docker
       containers run correctly?")
5      code_hard_version: bool = Field(default=False, description="Does the
       generated code use vulnerable version of the 'HARD' services?")
6      network_setup: bool = Field(default=False, description="Are all services
       /containers setup to be accessible from the right network ports?")
7      exploit_surface_ok: bool = Field(default=False, description="Are the
       services correctly responding to the probes?")
8      flag_placed: bool = Field(default=False, description="Is the flag placed
       at the intended location?")
9      flag_protected: bool = Field(default=False, description="Is the flag
       correctly protected from trivial attempts to reach it (e.g. 'curl http://localhost/
       flag.txt')?")
10     exploit_success: bool = Field(default=False, description="Was the flag
       captured during the exploit?")
```

Listing A.4: Definition of the milestones used in the pipeline

A.5 Stats

```
class Stats(BaseModel):
    num_containers: int = Field(default=0, description="Number of containers created while testing the Docker")
    test_iteration: int = Field(default=0, description="Number of iterations of the test loop")
    total_iterations: int = Field(default=0, description="Number of iterations considering possible resets after a Hard Fix by the debugger node")
    debug_retries: int = Field(default=0, description="Number of times the debugger node is reached")
    starting_image_builds: bool = Field(default=True, description="Checks if the LLM is able to generate a buildable Docker image in the first test iteration")
    image_build_failures: int = Field(default=0, description="Number of times the Docker image fails to build")
    starting_container_runs: bool = Field(default=True, description="Checks if the LLM is able to generate working Docker container in the first test iteration")
    container_run_failures: int = Field(default=0, description="Number of times the Docker container fails to run")
    not_vuln_version_fail: int = Field(default=0, description="Number of times the Docker builds and runs correctly but uses a not vulnerable version of the 'HARD' service")
    docker_misconfigured: int = Field(default=0, description="Number of times the Docker builds and runs correctly but uses a wrong network setup")
    docker_scout_vulnerable: bool = Field(default=False, description="Is the Docker environment vulnerable to the specified CVE?")
    exploitable: bool = Field(default=False, description="Does the exploit return the expected result?")
    services_ok: bool = Field(default=False, description="Does the Docker use all the services provided by the web search?")
    requires_manual_setup: bool = Field(default=False, description="Does this service require the user to perform some sort of manual operation to make it work or does it work just by launching the Docker?")
```

Listing A.5: Class used to create the `stats.json` file

A.6 DebuggerDecision

```
class DebuggerDecision(BaseModel):
    action_type: Literal["Soft Fix", "Hard Fix"] = Field(
```

```
3         description="'Soft Fix' if the agent made a tactical mistake, 'Hard Fix' if
      the Docker environment is broken"
4     )
5     analysis: str = Field(
6         description="One-sentence explanation of why the exploit failed"
7     )
8     expert_hint: str = Field(
9         description="For soft fix: exact instructions for the agent. For hard fix:
      explanation of what was broken"
10    )
11    fixed_code: Optional[Code] = Field(
12        default=None,
13        description="Corrected Docker code (required only if fix_type is 'hard'),
      null otherwise"
14    )
```

Listing A.6: Class used to define the possible decision types the *Debugger* can choose

Bibliography

[1] Ullah et al. *From CVE Entries to Verifiable Exploits: An Automated Multi-Agent Framework for Reproducing CVEs*. 2025. URL: https://arxiv.org/abs/2509.01835 (cit. on pp. 1, 21, 80).

[2] Docker. *Docker Scout*. URL: https://docs.docker.com/scout/ (cit. on pp. 2, 19).

[3] Gabriele Sannia. *Design of an AI Agent for the Generation of Vulnerable Virtual Environments*. 2025. URL: https://webthesis.biblio.polito.it/id/eprint/38698 (cit. on pp. 3, 23, 80).

[4] NIST. *The National Institute of Standards and Technology*. URL: https://www.nist.gov/ (cit. on p. 5).

[5] NIST. *NIST definition for the term vulnerability used in their glossary*. URL: https://csrc.nist.gov/glossary/term/vulnerability (cit. on p. 5).

[6] NIST. *The National Vulnerability Database*. URL: https://nvd.nist.gov/ (cit. on p. 5).

[7] NIST. *NIST vulnerability definition adopted in the NVD (National Vulnerability Database)*. URL: https://nvd.nist.gov/vuln (cit. on p. 5).

[8] NIST. *NIST's definition of the term threat*. URL: https://csrc.nist.gov/glossary/term/threat (cit. on p. 5).

[9] NIST. *NIST's definition of the term attack*. URL: https://csrc.nist.gov/glossary/term/attack (cit. on p. 6).

[10] NIST. *NIST's definition of the term weakness*. URL: https://csrc.nist.gov/glossary/term/weakness (cit. on p. 6).

[11] NIST. *NIST's definition of the term confidentiality*. URL: https://csrc.nist.gov/glossary/term/confidentiality (cit. on p. 6).

[12] NIST. *NIST's definition of the term integrity*. URL: https://csrc.nist.gov/glossary/term/integrity (cit. on p. 6).

[13] NIST. *NIST's definition of the term availability*. URL: https://csrc.nist.gov/glossary/term/availability (cit. on p. 6).

[14] MITRE. *Common Vulnerabilities and Exposures*. URL: https://www.cve.org/ (cit. on p. 6).

[15] MITRE. *The MITRE organization*. URL: https://www.mitre.org/ (cit. on p. 7).

[16] MITRE. *Common Weakness Enumeration*. URL: https://cwe.mitre.org/ (cit. on p. 7).

[17] FIRST. *Common Vulnerability Scoring System*. URL: https://www.first.org/cvss/ (cit. on p. 7).

[18] FIRST. *Exploit Prediction Scoring System*. URL: https://www.first.org/epss/ (cit. on p. 8).

[19] Cisco. *Cisco's definition of the term exploit*. URL: https://www.cisco.com/site/us/en/learn/topics/security/what-is-an-exploit.html (cit. on p. 8).

[20] Lockheed Martin. *The Cyber Kill Chain*. URL: https://www.lockheedmartin.com/en-us/capabilities/cyber/cyber-kill-chain.html (cit. on p. 9).

[21] Docker. *Docker*. URL: https://www.docker.com/ (cit. on p. 13).

[22] Vaswani et al. *Attention is all you need*. 2017. URL: https://arxiv.org/abs/1706.03762 (cit. on p. 16).

[23] Brown et al. *Language models are few-shot learners*. 2020. URL: https://arxiv.org/abs/2005.14165 (cit. on p. 17).

[24] Wei et al. *Finetuned models are zero-shot learners*. 2022. URL: https://arxiv.org/abs/2109.01652 (cit. on p. 17).

[25] Vulhub. *Vulhub*. URL: https://vulhub.org/ (cit. on pp. 20, 43, 84).

[26] Shao et al. *NYU CTF Bench: A Scalable Open-Source Benchmark Dataset for Evaluating LLMs in Offensive Security*. 2024. URL: https://arxiv.org/abs/2406.05590 (cit. on p. 22).

[27] Zhang et al. *Cybench: A Framework for Evaluating Cybersecurity Capabilities and Risks of Language Models*. 2024. URL: https://arxiv.org/abs/2408.08926 (cit. on p. 22).

[28] Zhu et al. *CVE-Bench: A Benchmark for AI Agents' Ability to Exploit Real-World Web Application Vulnerabilities*. 2025. URL: https://arxiv.org/abs/2503.17332 (cit. on p. 22).

[29] Julius Henke. *AutoPentest: Enhancing Vulnerability Management With Autonomous LLM Agents*. 2025. URL: https://arxiv.org/abs/2505.10321 (cit. on p. 22).

[30] Deng et al. *PentestGPT: An LLM-empowered Automatic Penetration Testing Tool*. 2023. URL: https://arxiv.org/abs/2308.06782 (cit. on p. 22).

[31] LangChain. *LangGraph*. URL: https://www.langchain.com/langgraph (cit. on pp. 27, 45).

[32] OpenCode. *OpenCode*. URL: https://opencode.ai/ (cit. on pp. 27, 45).

[33] Pydantic. *Pydantic*. URL: https://pydantic.dev/docs/validation/latest/get-started/ (cit. on pp. 29, 46).

[34] Offsec. *Exploit-DB*. URL: https://www.exploit-db.com/ (cit. on p. 43).

[35] Trickest. *Trickest CVE repository*. URL: https://github.com/trickest/cve (cit. on p. 43).

[36] GitHub. *GitHub*. URL: https://github.com/ (cit. on p. 43).

[37] High-Flyer. *Deepseek V4 Flash*. URL: https://openrouter.ai/deepseek/deepseek-v4-flash (cit. on p. 45).

[38] OpenRouter. *OpenRouter*. URL: https://openrouter.ai/ (cit. on p. 46).